

AWS IAM & CloudWatch

Assignment Report & Analysis

IAM Users, Groups, Policies & Roles • CloudWatch Alarms, SNS & Dashboards

Services: IAM, STS, EC2, CloudWatch, SNS

Work performed: 27 August – 26 September 2026

Contents

1. Overview & Key Concepts
2. Assignment 1 — IAM Users & Groups
3. Assignment 2 — Custom IAM Policies
4. Assignment 3 — IAM Role for VPC & DynamoDB
5. Assignment 4 — CloudWatch Alarms & SNS Notifications
6. Assignment 5 — CloudWatch Dashboard
7. Observations & Recommendations

1. Overview & Key Concepts

1.1 Objective

These assignments cover two pillars of running workloads on AWS. Assignments 1–3 set up secure, role-based access for a development team and an operations team using AWS Identity and Access Management (IAM), following the principle of least privilege. Assignments 4–5 add monitoring with Amazon CloudWatch — alarms that notify by email through Amazon SNS, and a dashboard that visualises EC2 performance.

Note: The repository contains screenshots only (no task-question files). The task statements below were derived from the screenshots and the resources created.

1.2 Tasks

- **Assignment 1:** Create four IAM users (Dev1, Dev2, Test1, Test2) and two groups (Dev-Team, Ops-Team). Add Dev1 and Dev2 to Dev-Team; add Dev1, Test1 and Test2 to Ops-Team.
- **Assignment 2:** Create two customer-managed policies — XYZ-Policy-1 (S3 full, EC2 create, RDS full) for Dev-Team and XYZ-Policy-2 (CloudWatch & billing full, read-only S3/EC2) for Ops-Team — and attach them to the groups.
- **Assignment 3:** Create an IAM role with full VPC and DynamoDB access that only Dev1 and Dev2 can assume, and verify it by switching roles in the console.
- **Assignment 4:** Create a billing alarm above \$500 and an EC2 CPU-utilization alarm above 65% that emails an alert through an SNS topic.
- **Assignment 5:** Launch an EC2 instance and build a CloudWatch dashboard showing its CPU utilization and network in/out.

1.3 Key Concepts

A quick reference for the building blocks used throughout this report:

Concept	What it means
IAM User	A permanent identity for one person or application, with its own sign-in password and/or access keys.
IAM Group	A collection of users. Policies attached to a group apply to every member, so permissions are managed once per team rather than per person.
IAM Policy	A JSON document that lists which actions are Allowed or Denied on which resources. "Customer managed" policies are ones you write; "AWS managed" ones are provided by AWS.
IAM Role	An identity with permissions but no permanent credentials. Trusted users or services "assume" it and get temporary credentials.
Trust policy	Attached to a role; defines WHO is allowed to assume the role (the Principal).
sts:AssumeRole	The AWS Security Token Service action that hands out temporary credentials for a role. A user needs permission to call it.
Least privilege	Grant only the minimum permissions required. Anything not explicitly allowed is denied by default (implicit deny).
CloudWatch metric	A time-ordered set of data points, e.g. an EC2 instance's CPUUtilization every 5 minutes.
CloudWatch alarm	Watches one metric against a threshold and moves between OK, ALARM and INSUFFICIENT_DATA states; can trigger actions such as an SNS

Concept	What it means
	notification.
SNS topic / subscription	Amazon Simple Notification Service. A topic receives messages; each confirmed subscription (email, SMS, Lambda...) gets a copy.
CloudWatch dashboard	A customisable page of graphs and widgets that shows metrics from one or more resources in one place.

1.4 Results at a Glance

Requirement	Status	Evidence
A1 Create 4 IAM users	Complete	Figure 1
A1 Create Dev-Team and Ops-Team with members	Complete	Figures 1–3
A2 Create XYZ-Policy-1 (Dev)	Complete	Figures 4–5
A2 Create XYZ-Policy-2 (Ops)	Complete	Figure 6
A2 Attach policies to groups	Complete	Figures 7–9
A3 Trust policy for Dev1 & Dev2	Complete	Figure 10
A3 Role with VPC + DynamoDB full access	Complete	Figures 11, 17
A3 Permission for users to assume role	Complete	Figures 12–16, 18
A3 Verify via Switch Role	Complete	Figures 19–22
A3 Negative test (no role = denied)	Complete	Figures 23–24
A4 Billing alarm > \$500	Complete	Figures 25–26
A4 EC2 instance + CPU alarm > 65%	Complete	Figures 27, 33–34
A4 SNS topic + confirmed email	Complete	Figures 28–32, 35
A5 EC2 instance for monitoring	Complete	Figures 36–37
A5 Dashboard: CPU, NetworkIn/Out	Complete	Figures 38–39

2. Assignment 1 — IAM Users & Groups

2.1 Tasks

1. Create IAM users Dev1, Dev2, Test1 and Test2.
2. Create user groups Dev-Team and Ops-Team.
3. Add Dev1 and Dev2 to Dev-Team; add Dev1, Test1 and Test2 to Ops-Team.

2.2 Step-by-Step Implementation

Step 1: Create the IAM users

Go to **IAM** → **Users** → **Create user**. For each of Dev1, Dev2, Test1 and Test2: enter the user name, tick **Provide user access to the AWS Management Console**, set a password (optionally forcing a reset at next sign-in), and click **Create user**.

Then create the groups under **IAM** → **User groups** → **Create group**: first **Dev-Team**, then **Ops-Team**. The banner confirms Ops-Team was created, and the user list shows all four new users.

Understanding: Users are individual identities. Each person should have their own user so that actions can be traced back to them in CloudTrail — never share credentials.

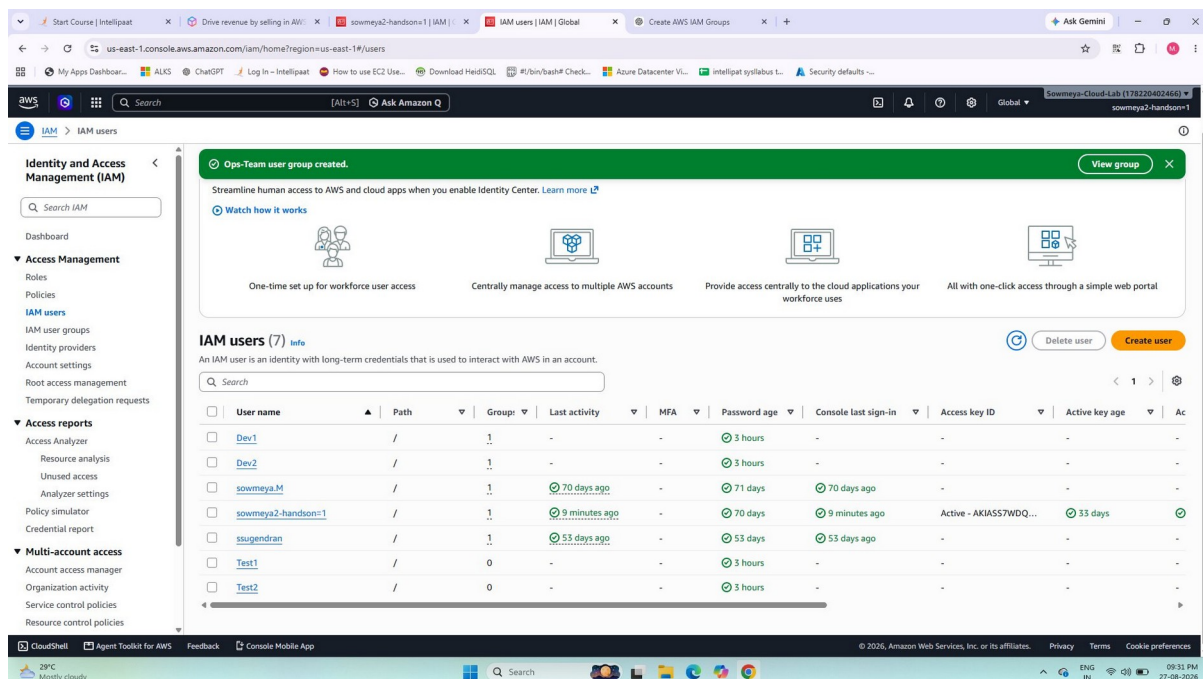


Figure 1: IAM users list — Dev1, Dev2, Test1, Test2 created; Ops-Team group created

Step 2: Add members to Dev-Team

Open **User groups** → **Dev-Team** → **Users** → **Add users**, select **Dev1** and **Dev2**, and click **Add users**. Dev-Team now has 2 users.

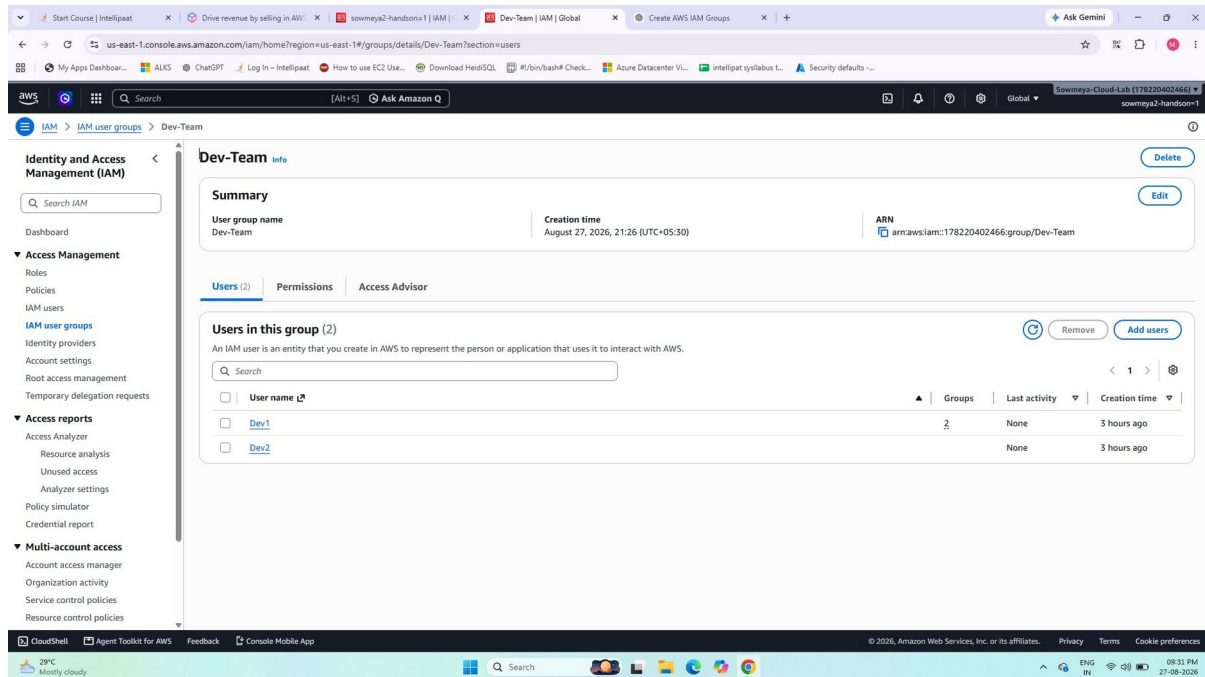


Figure 2: Dev-Team group containing Dev1 and Dev2

Step 3: Add members to Ops-Team

Open **Ops-Team** → **Add users**, select **Dev1**, **Test1** and **Test2**. Ops-Team now has 3 users. Dev1 shows "2" in the Groups column because it belongs to both teams.

Understanding: A user can belong to multiple groups and receives the combined permissions of all of them. Dev1 will therefore inherit both the developer and operations permissions in Assignment 2.

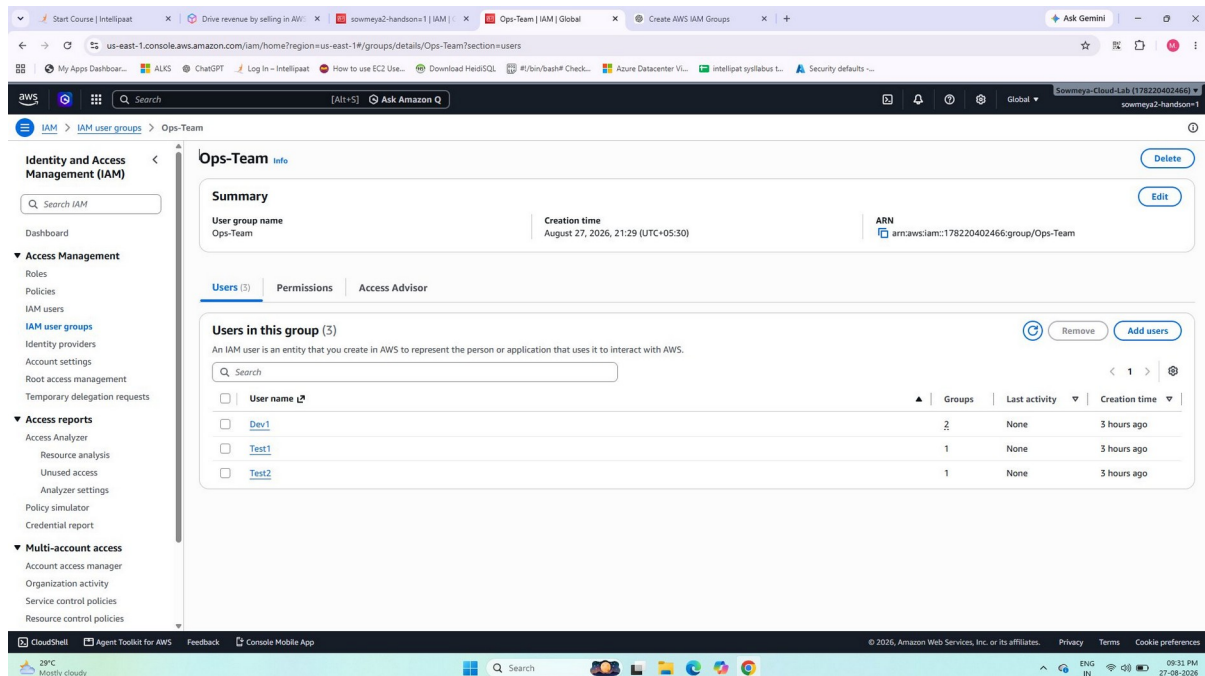


Figure 3: Ops-Team group containing Dev1, Test1 and Test2

2.3 Analysis

All users and groups exist with the correct membership. Organising users into groups means permissions in Assignment 2 can be granted once per team instead of repeated for every user — the AWS-recommended approach.

3. Assignment 2 — Custom IAM Policies

3.1 Tasks

1. Create XYZ-Policy-1: S3 full access, EC2 create access and RDS full access.
2. Create XYZ-Policy-2: CloudWatch and billing full access, with limited (read/list) S3 and EC2 access.
3. Attach XYZ-Policy-1 to Dev-Team and XYZ-Policy-2 to Ops-Team.

3.2 Step-by-Step Implementation

Step 1: Create XYZ-Policy-1 (Developer policy)

Go to **IAM** → **Policies** → **Create policy**. Using the Visual editor, add three services:

- **S3** — All S3 actions (Full access)
- **EC2** — Write actions such as RunInstances / CreateTags (Limited: Write = "create" access)
- **RDS** — All RDS actions (Full access)

Resources: All resources. Name it **XYZ-Policy-1** with description "S3 full access, EC2 create access, and RDS full access", then click **Create policy**.

Understanding: "Limited: Write" means only some write-level actions are allowed. Developers can launch/create EC2 resources but have not been given every EC2 action (for example, listing or reading may be excluded).

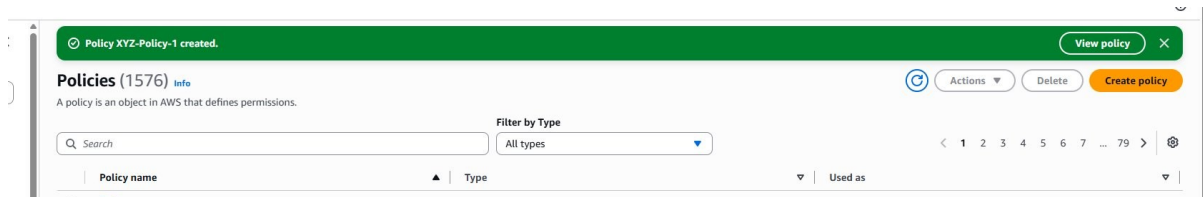


Figure 4: Policy XYZ-Policy-1 created successfully

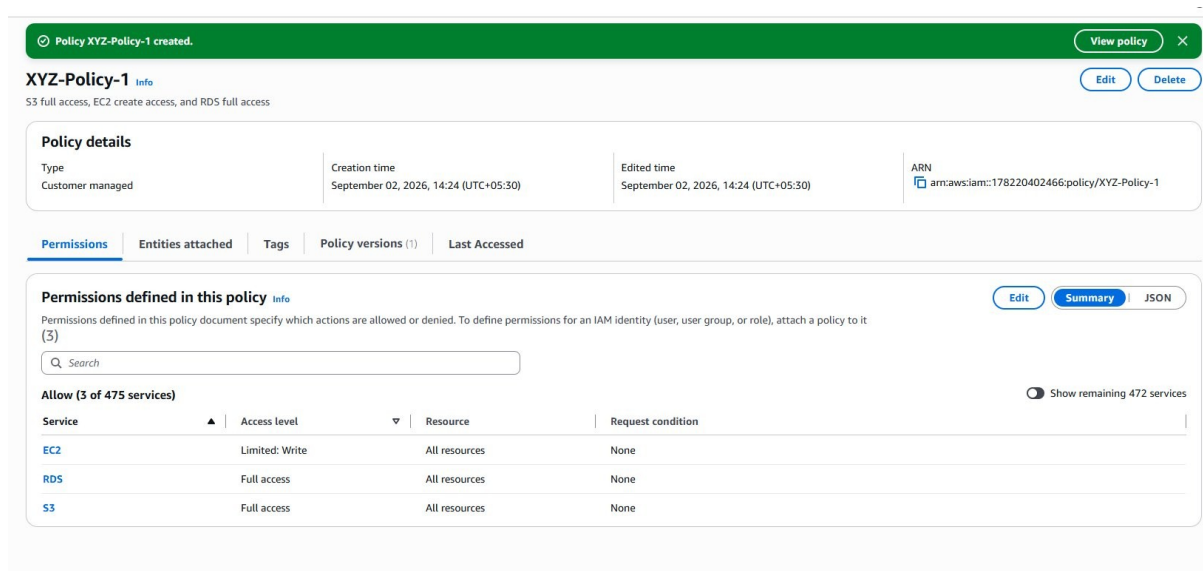


Figure 5: XYZ-Policy-1 permissions — EC2 (Limited: Write), RDS (Full), S3 (Full)

Step 2: Create XYZ-Policy-2 (Operations policy)

Create a second policy named **XYZ-Policy-2** with these permissions:

- **Full access:** Billing Console, Budget, CloudWatch, Cost and Usage Report, Cost Explorer Service
- **EC2** — Limited: List (view instances only)
- **S3** — Limited: List, Read (view buckets/objects only)

Understanding: The operations team monitors cost and health, so it gets full access to monitoring/billing tools but only read-only visibility into EC2 and S3 — it can observe resources but not change them.

Policy XYZ-Policy-2 created.

Policy details

Type Customer managed	Creation time September 02, 2026, 14:26 (UTC+05:30)	Edited time September 02, 2026, 14:26 (UTC+05:30)	ARN arn:aws:iam::123456789012:policy/XYZ-Policy-2
--------------------------	--	--	--

Permissions | Entities attached | Tags | Policy versions (1) | Last Accessed

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it (7)

Allow (7 of 475 services)

Service	Access level	Resource	Request condition
Billing Console	Full access	All resources	None
Budget	Full access	All resources	None
CloudWatch	Full access	All resources	None
Cost and Usage Report	Full access	All resources	None
Cost Explorer Service	Full access	All resources	None
EC2	Limited: List	All resources	None
S3	Limited: List, Read	All resources	None

Figure 6: XYZ-Policy-2 permissions — billing/monitoring full, EC2 list, S3 list/read

Step 3: Attach XYZ-Policy-1 to Dev-Team

Open **User groups** → **Dev-Team** → **Permissions** → **Add permissions** → **Attach policies**. Search "xyz", tick **XYZ-Policy-1** and click **Attach policies**.

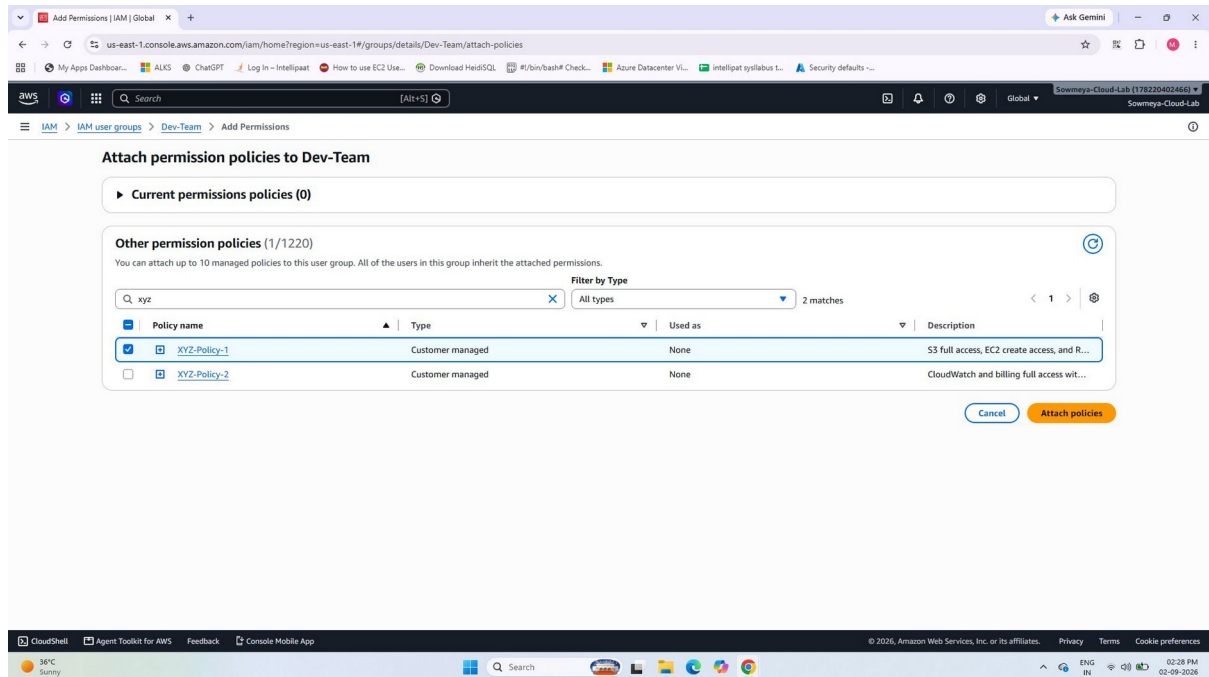


Figure 7: Selecting XYZ-Policy-1 to attach to Dev-Team

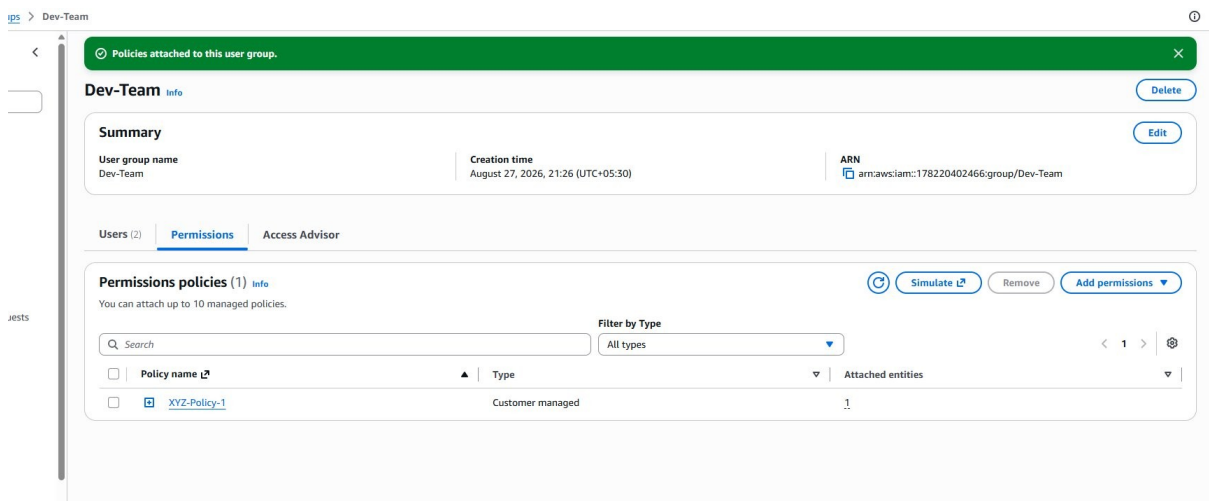


Figure 8: Dev-Team permissions — XYZ-Policy-1 attached

Step 4: Attach XYZ-Policy-2 to Ops-Team

Repeat for **Ops-Team**, attaching **XYZ-Policy-2**.

Understanding: Every member of a group inherits the group's policies automatically. Adding a new developer later only requires adding them to Dev-Team — no policy changes needed.

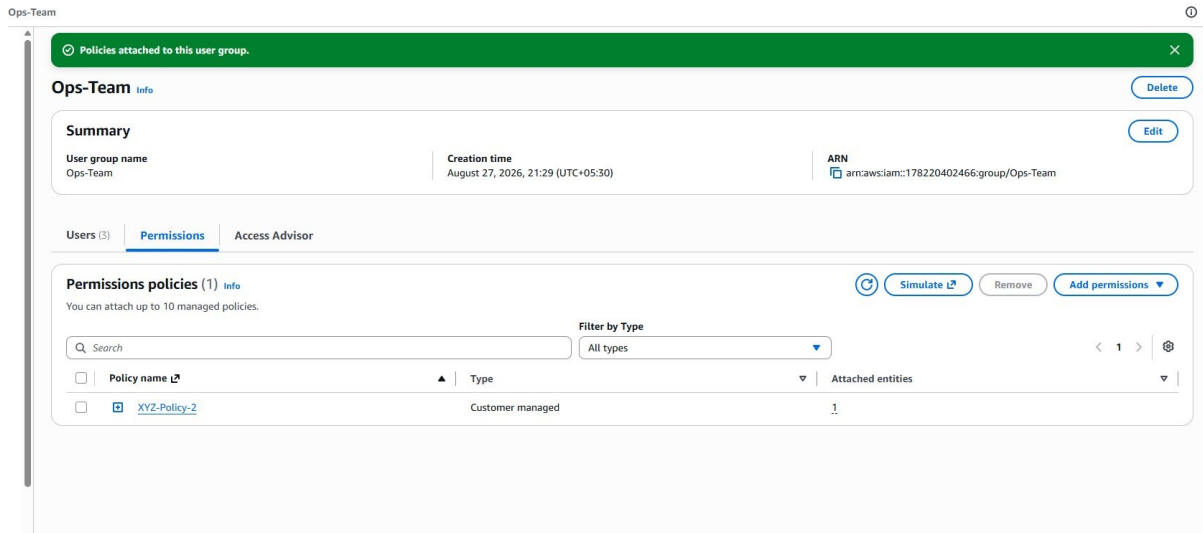


Figure 9: Ops-Team permissions — XYZ-Policy-2 attached

3.3 Analysis

Both policies were created as customer-managed policies with the intended access levels and attached to the correct groups. The effective permissions are: Dev2 → XYZ-Policy-1; Test1 and Test2 → XYZ-Policy-2; Dev1 → both policies (because it is in both groups, as confirmed later in Figure 15).

4. Assignment 3 — IAM Role for VPC & DynamoDB

4.1 Tasks

1. Create an IAM role with full access to Amazon VPC and Amazon DynamoDB.
2. Allow only Dev1 and Dev2 to assume the role.
3. Verify that the developers can switch into the role and use VPC/DynamoDB, and that access is denied without the role.

4.2 How role assumption works

Two policies must line up for a user to assume a role:

1. **Trust policy (on the role):** "Dev1 and Dev2 are allowed to assume me."
2. **Permission policy (on the user):** "This user is allowed to call sts:AssumeRole on XYZ-VPC-DynamoDB-Role."

When the user switches role, AWS STS issues temporary credentials. While in the role, the user has only the role's permissions (VPC + DynamoDB), not their normal user permissions.

4.3 Step-by-Step Implementation

Step 1: Create the role with a custom trust policy

Go to **IAM** → **Roles** → **Create role**, choose **Custom trust policy** and enter the JSON below. It names Dev1 and Dev2 as the only trusted principals.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::178220402466:user/Dev1",
          "arn:aws:iam::178220402466:user/Dev2"
        ]
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Understanding: IAM groups cannot be used as a Principal in a trust policy, which is why the individual user ARNs of the Dev-Team members are listed.

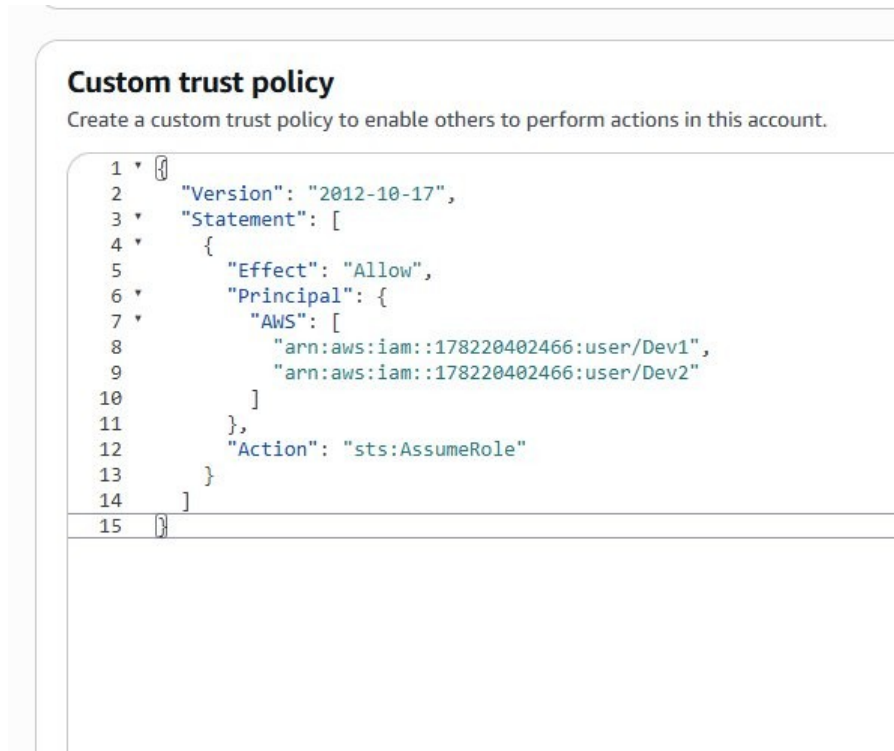


Figure 10: Custom trust policy allowing Dev1 and Dev2 to assume the role

Step 2: Attach VPC and DynamoDB permissions to the role

On the permissions step, attach the AWS managed policies **AmazonVPCFullAccess** and **AmazonDynamoDBFullAccess**, name the role, and create it. The role summary shows the ARN, a "Link to switch roles in console", and a 1-hour maximum session duration.

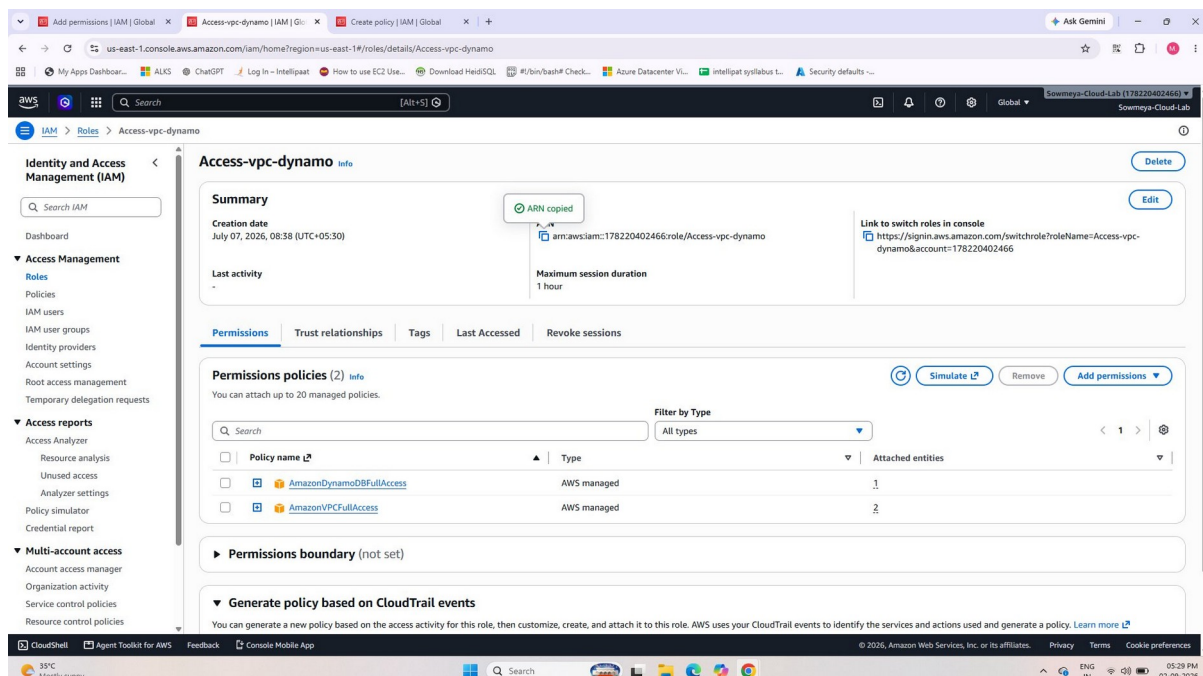


Figure 11: Role permissions — AmazonDynamoDBFullAccess and AmazonVPCFullAccess

Step 3: Create the AllowAssume permission policy

Go to **Policies** → **Create policy**. Choose service **STS**, action **AssumeRole**, and restrict the resource to the role **XYZ-VPC-DynamoDB-Role**. Name it **AllowAssumeXYZ-VPC-DynamoDB-Role** and create it. Equivalent JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "sts:AssumeRole",
    "Resource": "arn:aws:iam::178220402466:role/XYZ-VPC-DynamoDB-Role"
  }]
}
```

Understanding: Restricting the Resource to one specific role ARN means these users can assume only this role — not any other role in the account.

The screenshot shows the 'Create policy' page in the AWS IAM console. The policy name is 'AllowAssumeXYZ-VPC-DynamoDB-Role'. The permissions section shows a single permission: 'STS' with 'Limited: Write' access level and 'RoleName string like [XYZ-VPC-DynamoDB-Role]' as the resource. The 'Add tags' section is empty.

Figure 12: Reviewing AllowAssumeXYZ-VPC-DynamoDB-Role — STS, Limited: Write

The screenshot shows the 'Policy details' page for the 'AllowAssumeXYZ-VPC-DynamoDB-Role' policy. The policy is 'Customer managed' and was created on 'September 02, 2026, 18:29 (UTC+05:30)'. The ARN is 'arn:aws:iam::178220402466:policy/AllowAssumeXYZ-VPC-DynamoDB-Role'. The permissions section shows a single permission: 'STS' with 'Limited: Write' access level and 'RoleName string like [XYZ-VPC-DynamoDB-Role]' as the resource.

Figure 13: AllowAssumeXYZ-VPC-DynamoDB-Role policy created

Step 4: Attach the AllowAssume policy to Dev1 and Dev2

Open IAM users → Dev1 → Add permissions → Attach policies directly, select AllowAssumeXYZ-VPC-DynamoDB-Role, click Next then Add permissions. Repeat for Dev2.

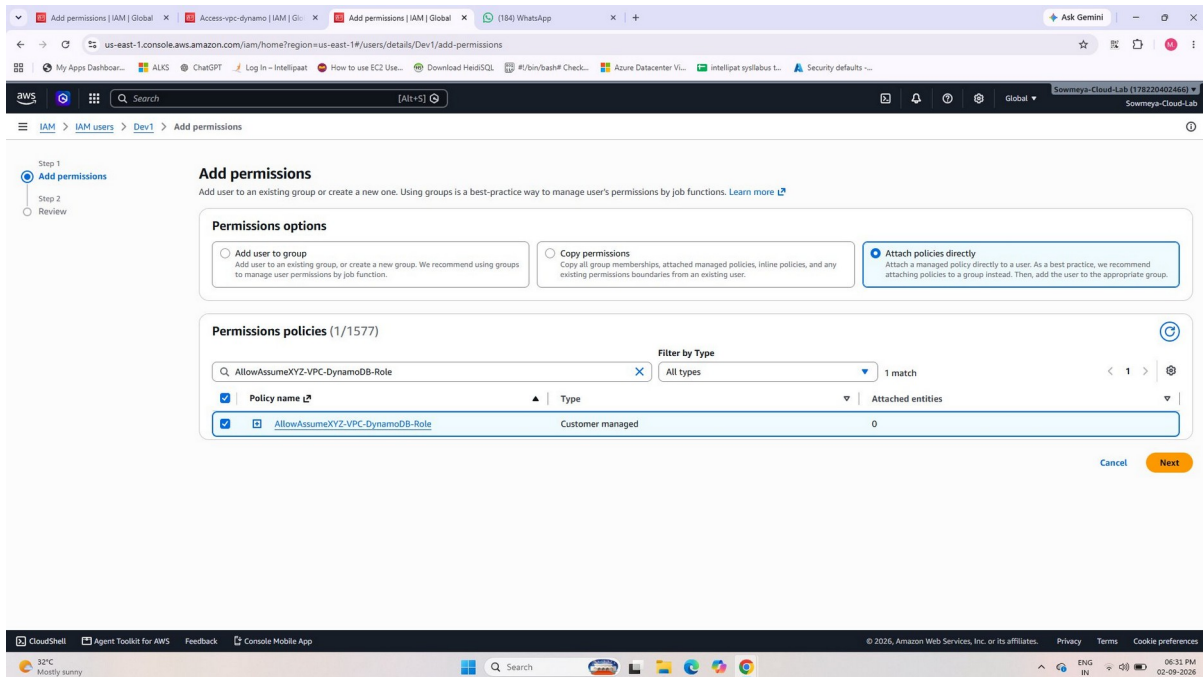


Figure 14: Attaching the AllowAssume policy directly to Dev1

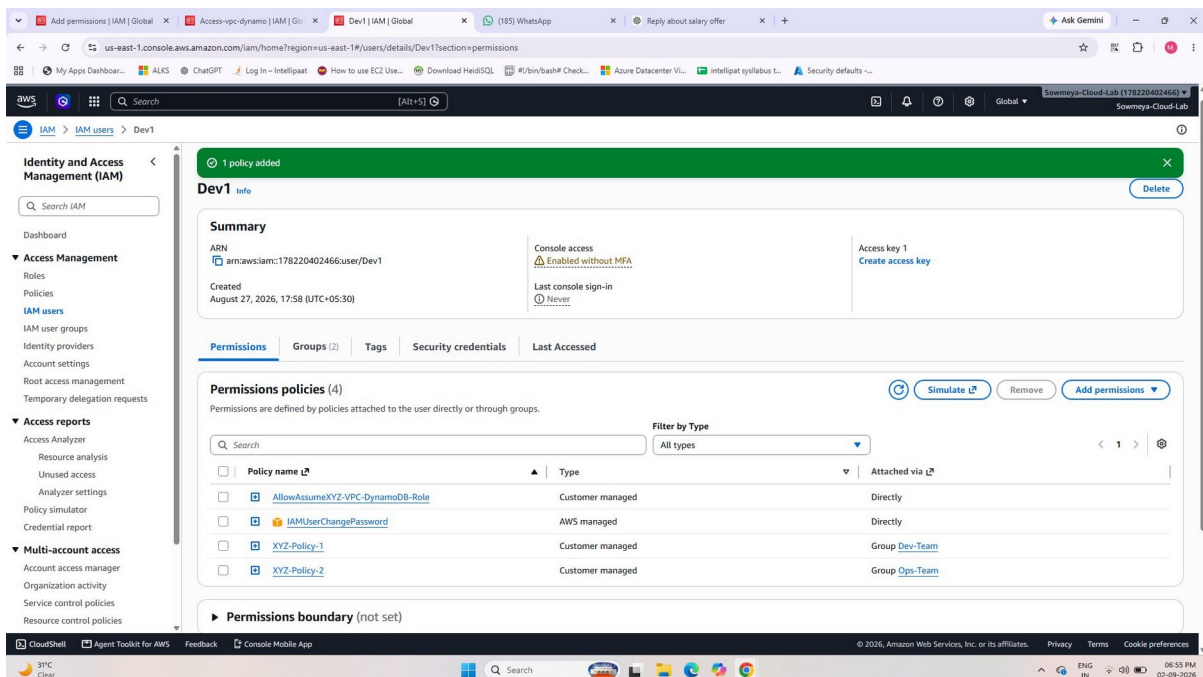


Figure 15: Dev1 effective permissions — AllowAssume (direct), XYZ-Policy-1 (Dev-Team), XYZ-Policy-2 (Ops-Team)

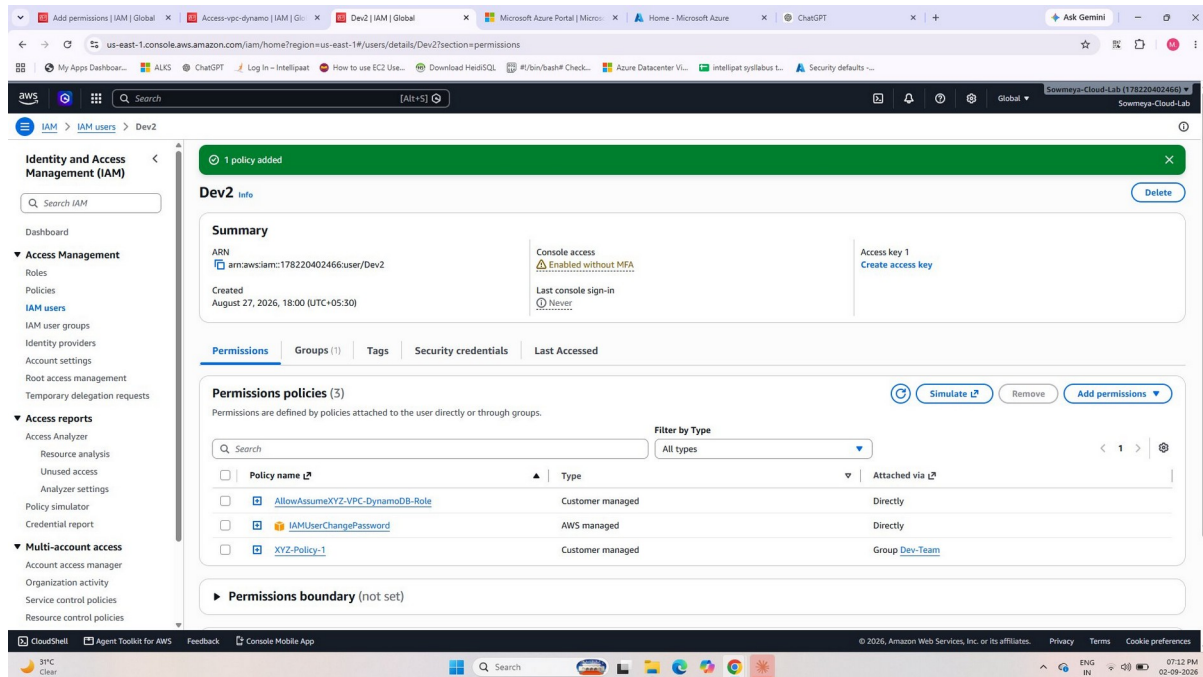


Figure 16: Dev2 effective permissions — AllowAssume (direct), XYZ-Policy-1 (Dev-Team)

Step 5: Confirm the role and policy

The Roles list shows XYZ-VPC-DynamoDB-Role with trusted entity "Account: 178220402466" (i.e. principals in this account). The AllowAssume policy summary confirms the STS permission scoped to that role.

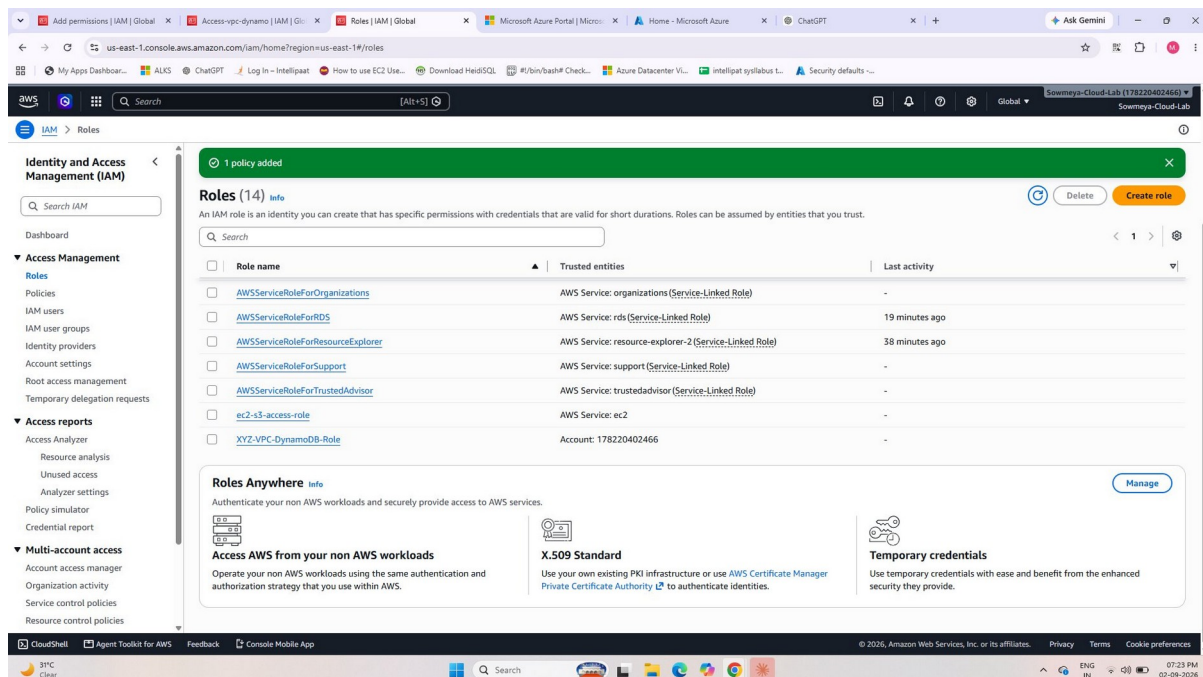


Figure 17: Roles list — XYZ-VPC-DynamoDB-Role trusted by the account

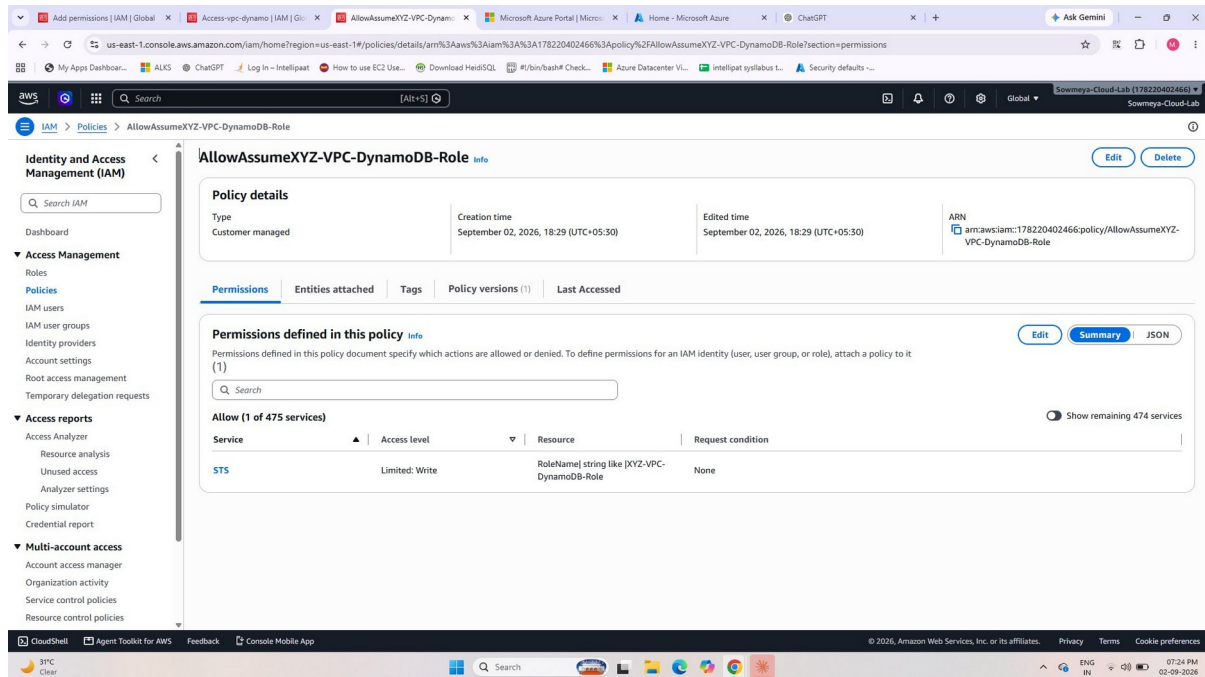


Figure 18: AllowAssumeXYZ-VPC-DynamoDB-Role policy summary

Step 6: Sign in as Dev1 and open Switch role

In a private/incognito window, sign in as **Dev1**. Normal access is limited (e.g. Cost and usage shows "Access denied"). Open the account menu (top right) and click **Switch role**.

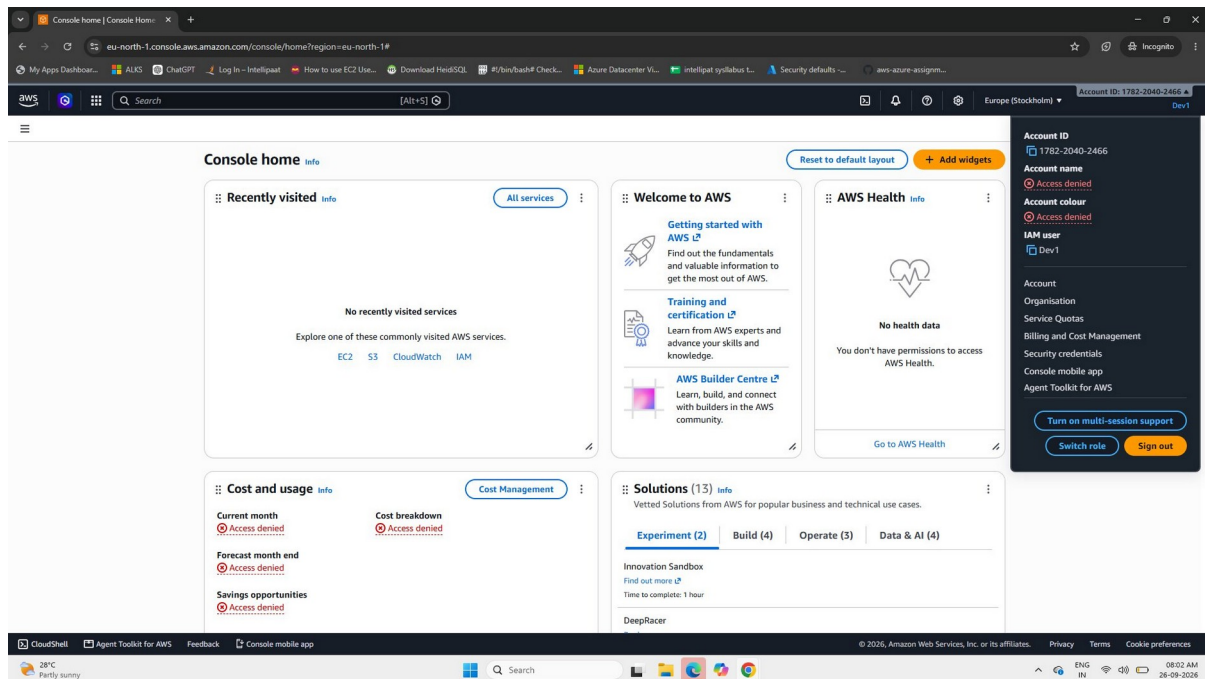


Figure 19: Signed in as IAM user Dev1 — Switch role option in the account menu

Step 7: Switch to XYZ-VPC-DynamoDB-Role

Enter Account ID **178220402466**, Role name **XYZ-VPC-DynamoDB-Role**, an optional display name and colour, then click **Switch role**. The header now shows **XYZ-VPC-DynamoDB** instead of Dev1, confirming the role is active.

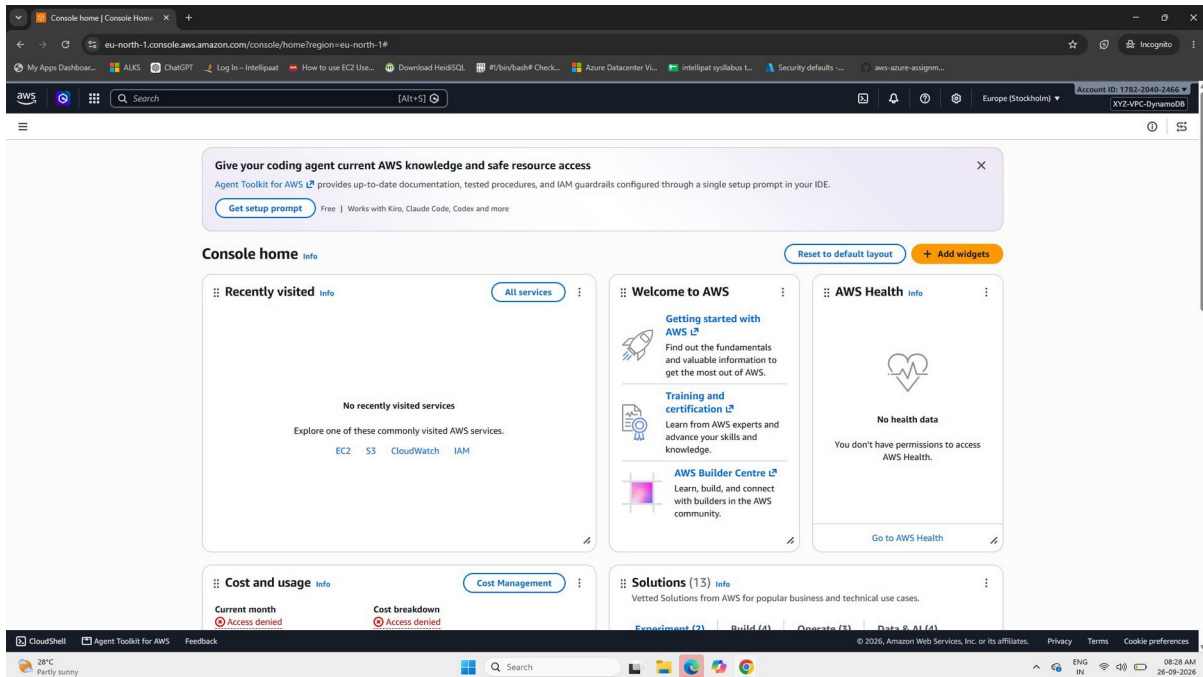


Figure 20: Console header shows the assumed role XYZ-VPC-DynamoDB

Step 8: Verify DynamoDB and VPC access using the role

While in the role, open DynamoDB and VPC. DynamoDB loads the Tables page without errors (no tables exist in eu-north-1 yet), and VPC lists the default VPC (172.31.0.0/16) with full details.

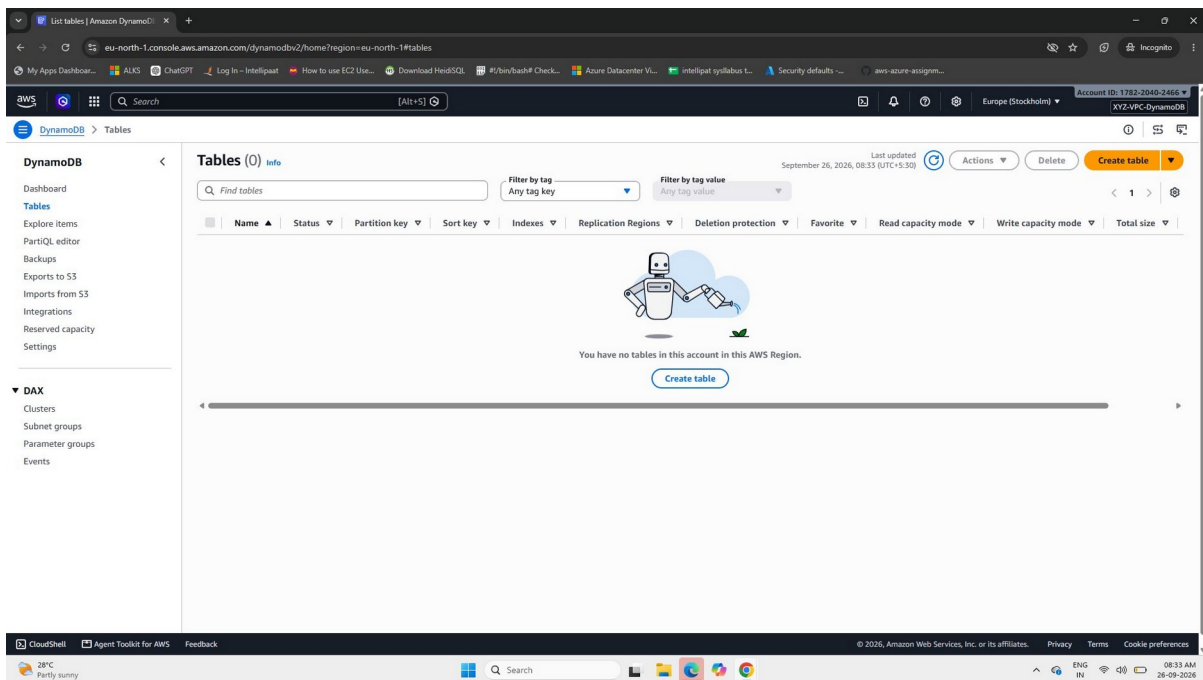


Figure 21: DynamoDB Tables page accessible through the role

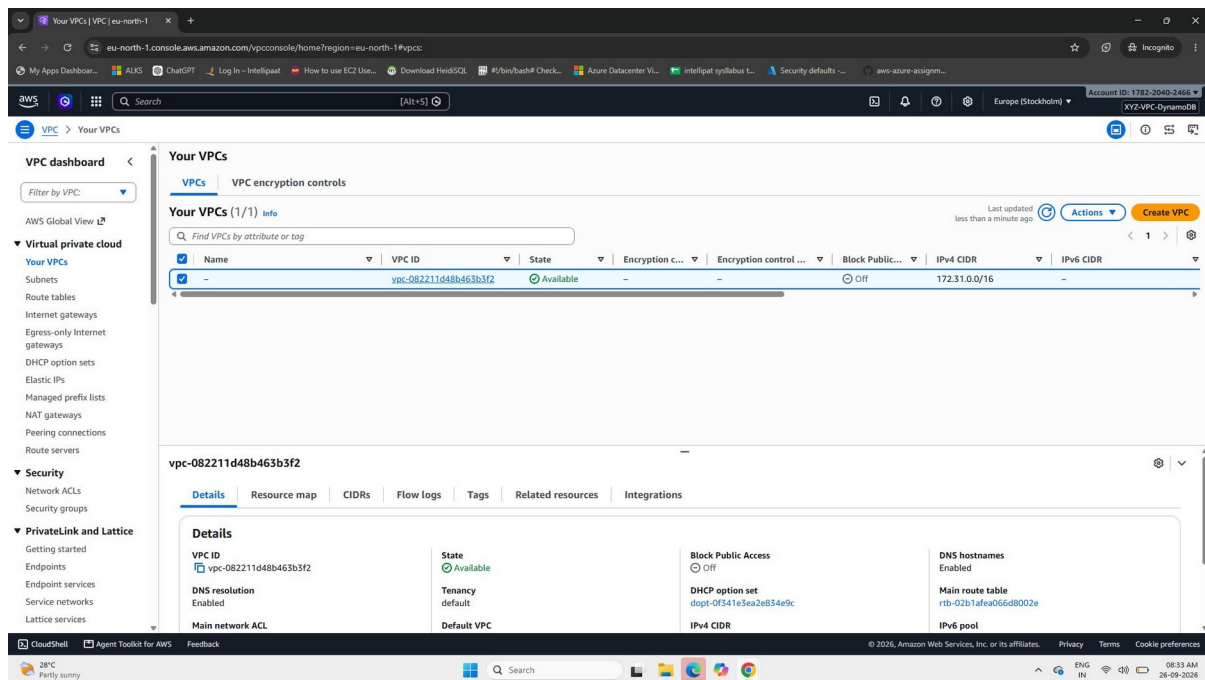


Figure 22: VPC console accessible through the role — default VPC listed

Step 9: Negative test — Dev2 without the role

Sign in as **Dev2** without switching roles and try the same services. DynamoDB returns "not authorized to perform: dynamodb:ListTables" and VPC returns "not authorized to perform: ec2:DescribeVpcs / ec2:DescribeSubnets", because no identity-based policy on Dev2 allows those actions.

Understanding: This proves the VPC/DynamoDB access comes only from the role. The user's own policies don't grant it, so access is available only when a developer deliberately assumes the role — and it is time-limited (1 hour).

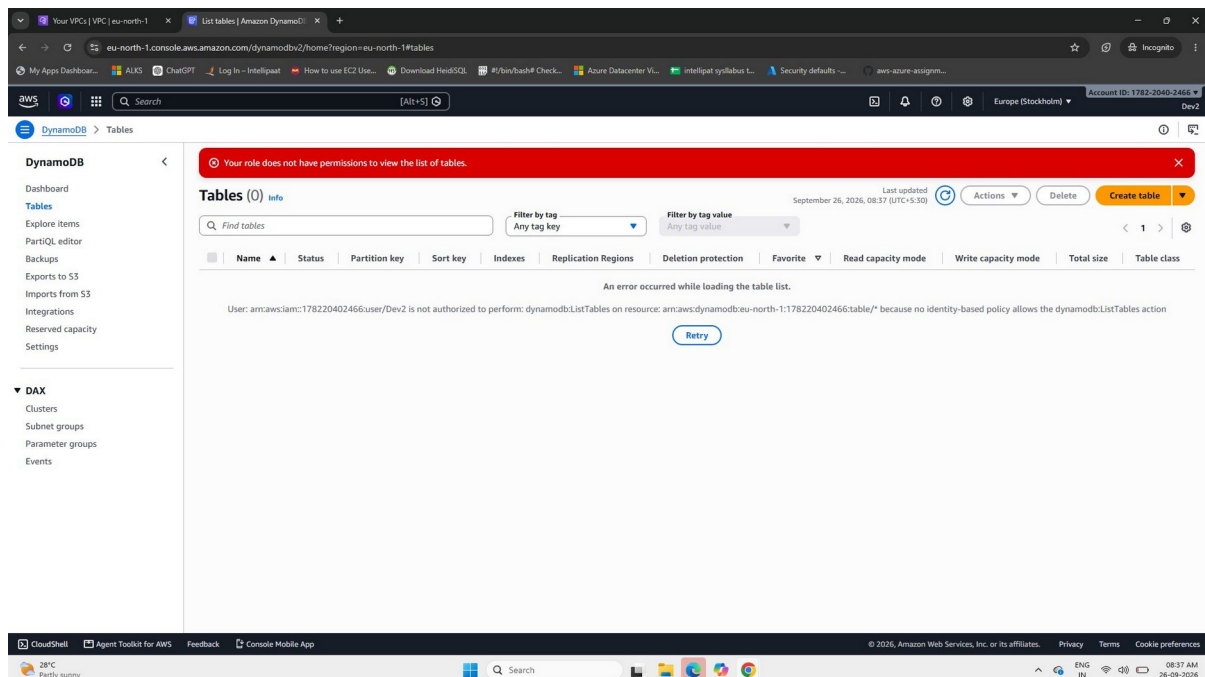


Figure 23: Dev2 (no role) — DynamoDB ListTables denied

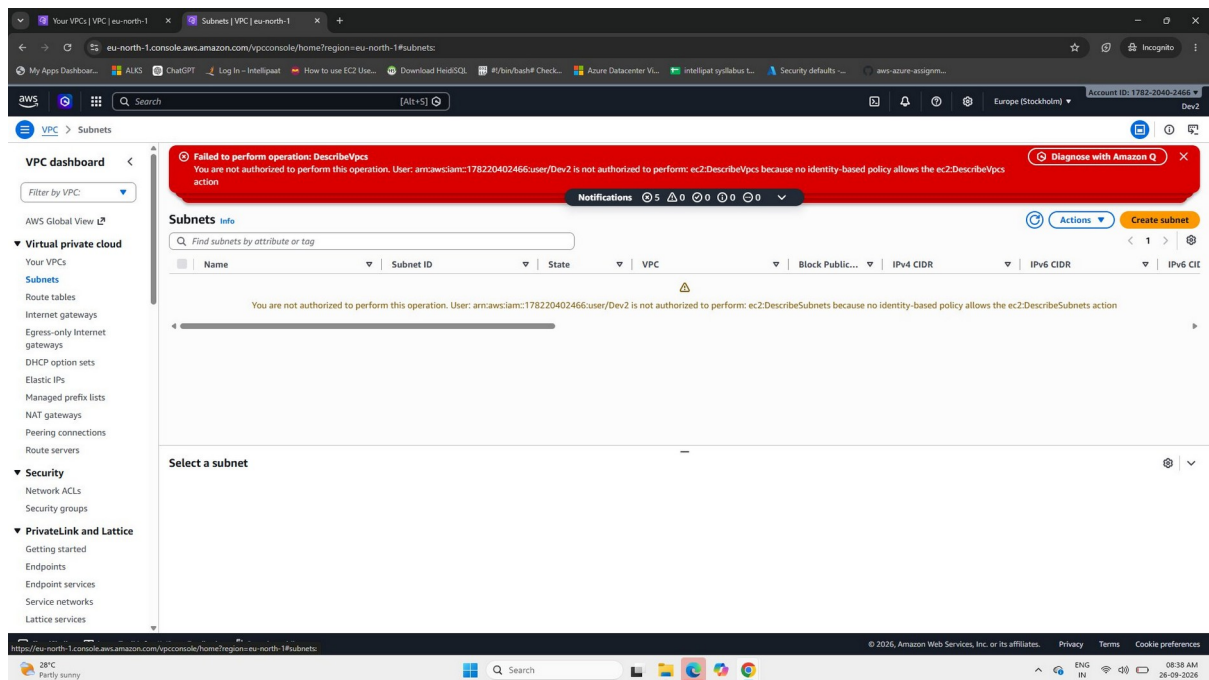


Figure 24: Dev2 (no role) — VPC DescribeVpcs / DescribeSubnets denied

4.4 Analysis

The role-assumption pattern is implemented end to end: a trust policy limited to Dev1 and Dev2, a tightly scoped sts:AssumeRole permission on each user, a successful Switch Role with working VPC and DynamoDB access, and a negative test showing access is denied without the role. This is a strong demonstration of least privilege.

5. Assignment 4 — CloudWatch Alarms & SNS Notifications

5.1 Tasks

1. Create a billing alarm that triggers when estimated charges exceed \$500.
2. Launch an EC2 instance and create a CloudWatch alarm for CPU utilization above 65%.
3. Send an email notification through Amazon SNS when the CPU alarm fires.

5.2 How the pieces connect

EC2 publishes the CPUUtilization metric to CloudWatch. A CloudWatch alarm watches that metric and changes state (OK → ALARM) when the threshold is crossed. The alarm's action publishes a message to an SNS topic, and SNS delivers it to every confirmed subscriber — here, an email address.

1. Source	2. Monitor	3. Notify	4. Receive
EC2 instance → CPUUtilization metric	CloudWatch alarm (CPU > 65%)	SNS topic EC2-CPU-Alerts	Email subscriber

5.3 Step-by-Step Implementation

Step 1: Create the billing alarm (Billing-Above-500)

Billing metrics exist only in **US East (N. Virginia) us-east-1**. Go to **CloudWatch → Alarms → Create alarm → Select metric → Billing → Total Estimated Charge**, choose **USD**, set the condition **Greater than 500**, name it **Billing-Above-500** and create it.

Understanding: The state shows "Insufficient data" at first because AWS updates the billing metric only a few times a day. It moves to OK once data arrives (see Figure 34).

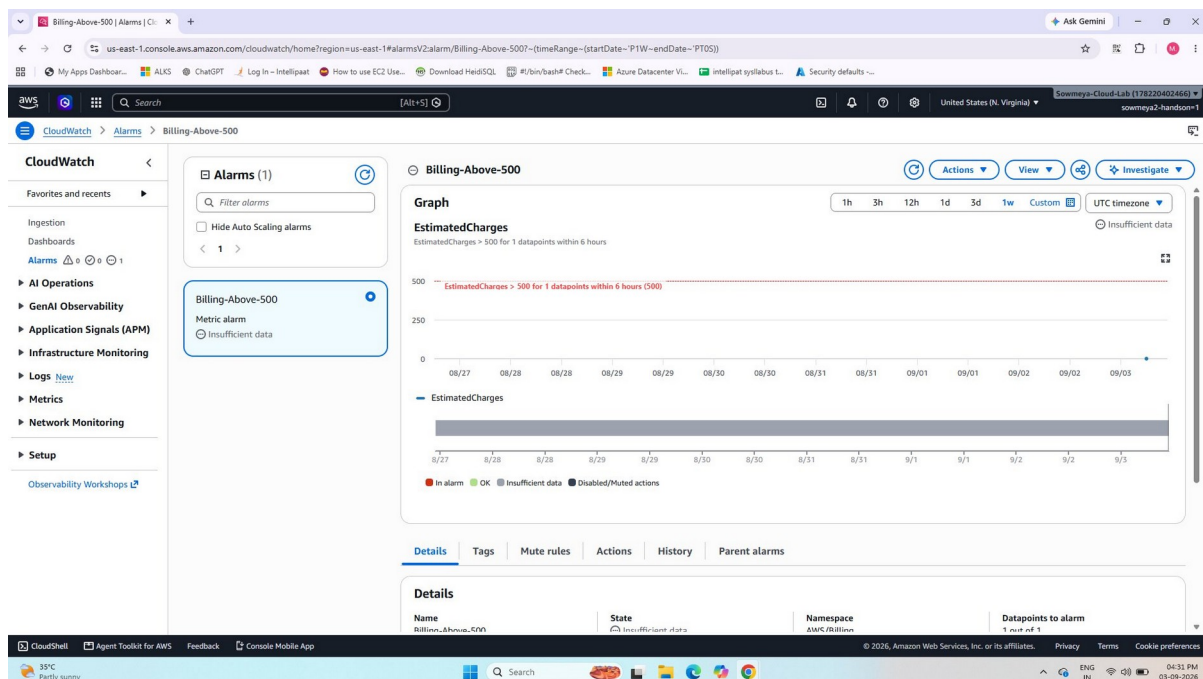


Figure 25: Billing-Above-500 alarm — EstimatedCharges threshold line at \$500

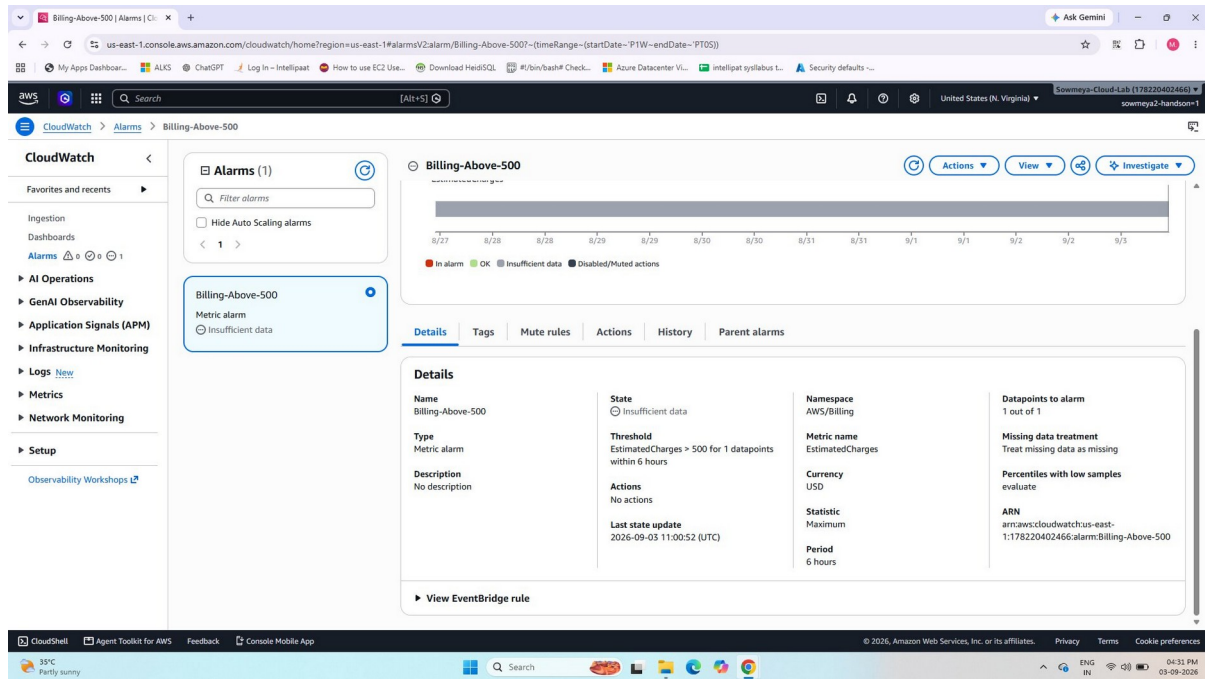


Figure 26: Billing alarm details — AWS/Billing, EstimatedCharges > 500, period 6 hours

Step 2: Launch the EC2 instance to monitor

Launch a **t3.micro** instance named **CPU-Monitoring-EC2** in **us-east-1**. Wait until it is **Running** with **3/3 status checks passed**. Note the Instance ID (**i-03e2a1fa9d3068437**) for the alarm.

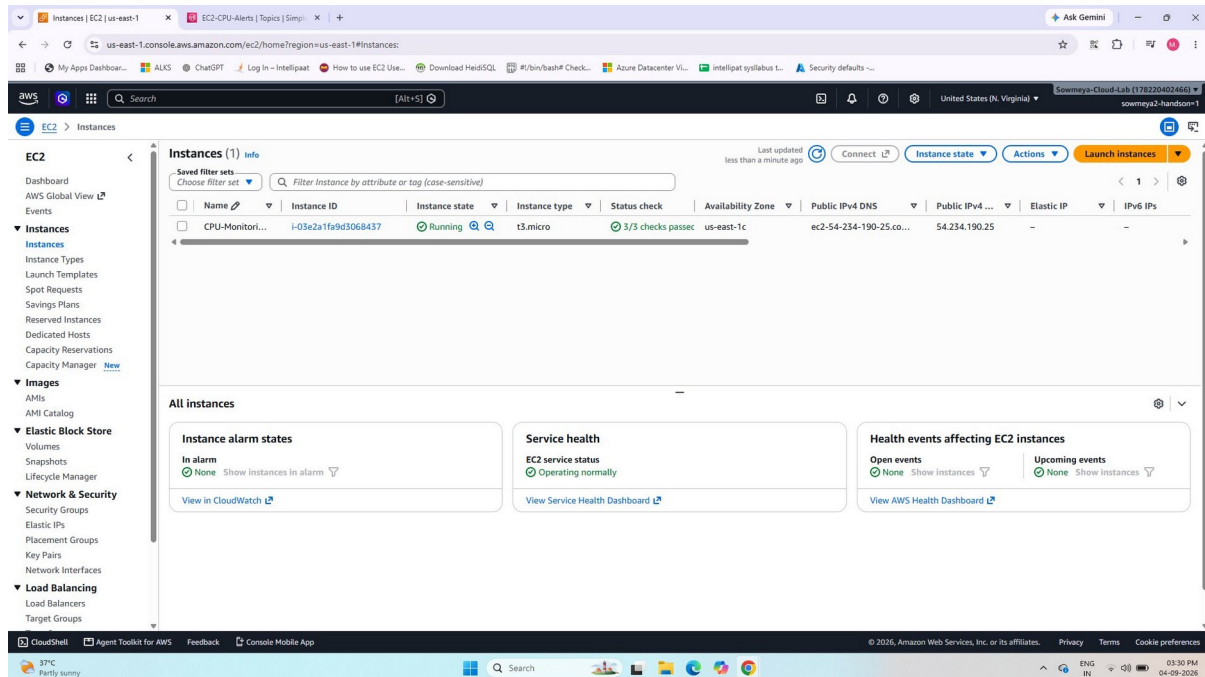


Figure 27: CPU-Monitoring-EC2 instance running in us-east-1c

Step 3: Create the SNS topic

Go to **Amazon SNS** → **Topics** → **Create topic**, choose type **Standard**, name it **EC2-CPU-Alerts** and click **Create topic**.

Understanding: A topic is a communication channel. Publishers (the alarm) send to the topic; SNS fans the message out to all subscribers, so you can add more emails, SMS or Lambda functions later without changing the alarm.

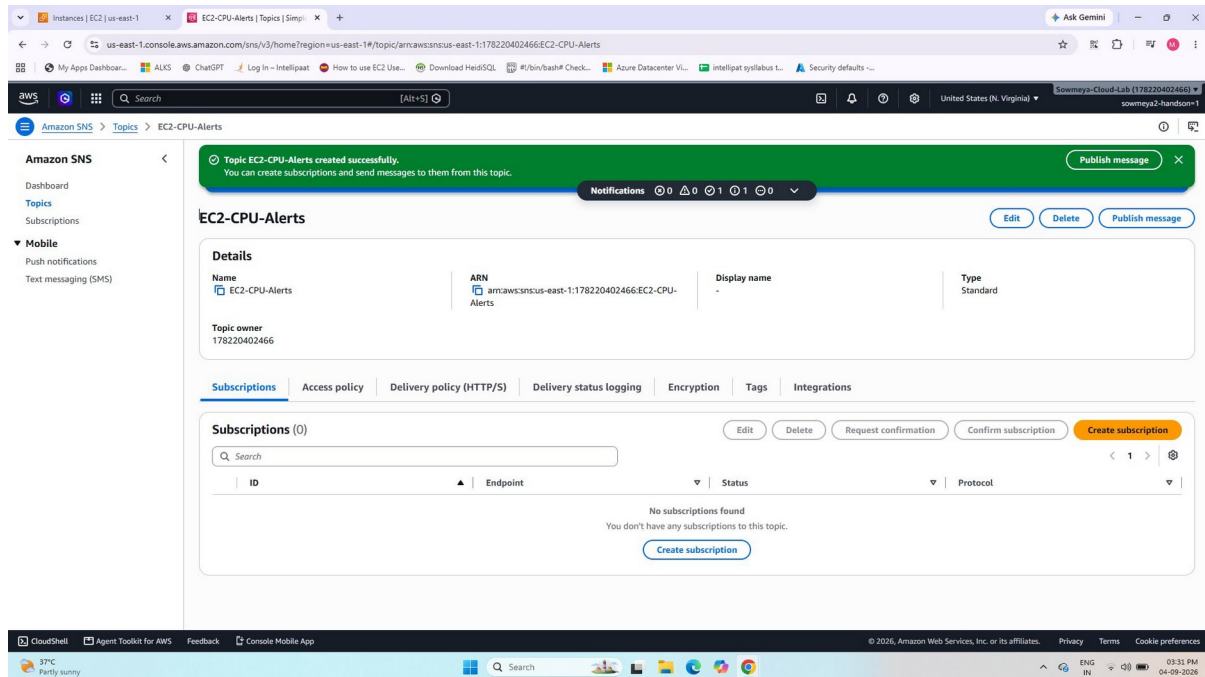


Figure 28: SNS topic EC2-CPU-Alerts created (no subscriptions yet)

Step 4: Create an email subscription

On the topic, click **Create subscription**, choose protocol **Email**, enter the endpoint email address, and create it. Its status is **Pending confirmation**.

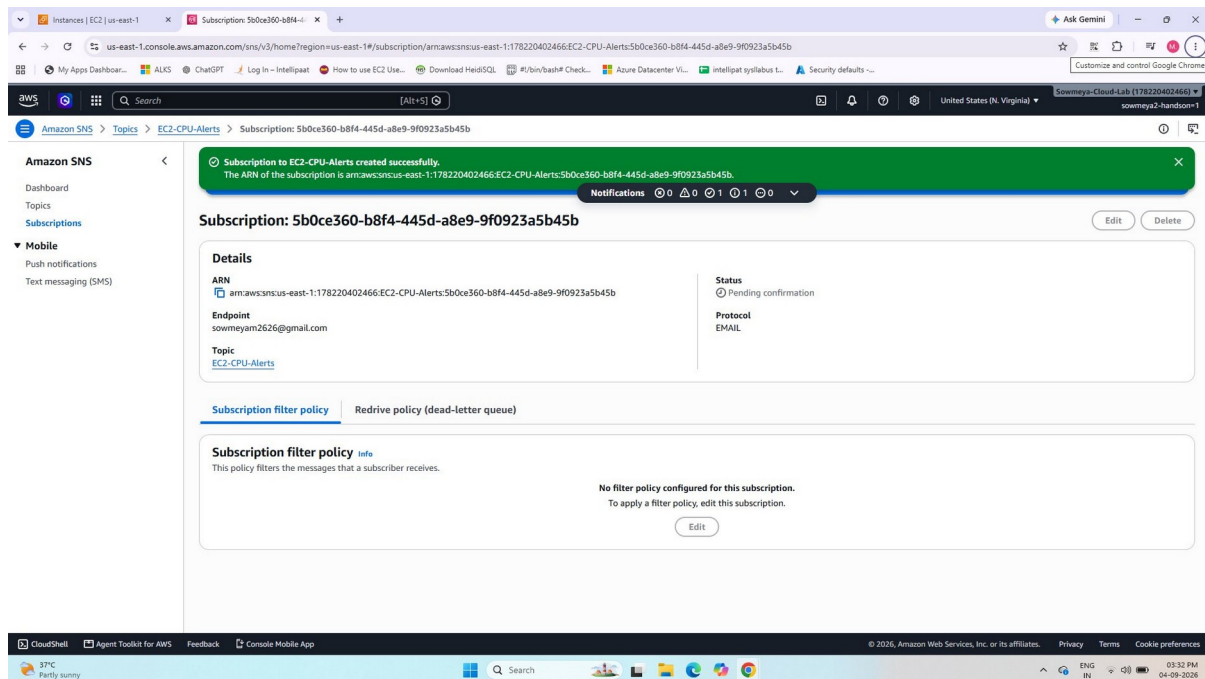


Figure 29: Email subscription created — Pending confirmation

Step 5: Confirm the subscription from the email

Open the "AWS Notification – Subscription Confirmation" email and click **Confirm subscription**. The browser shows "Subscription confirmed!", and the topic now lists the subscription as **Confirmed**.

Understanding: SNS will not send any messages to an email address until the owner confirms it. This prevents anyone from subscribing someone else's inbox without permission.

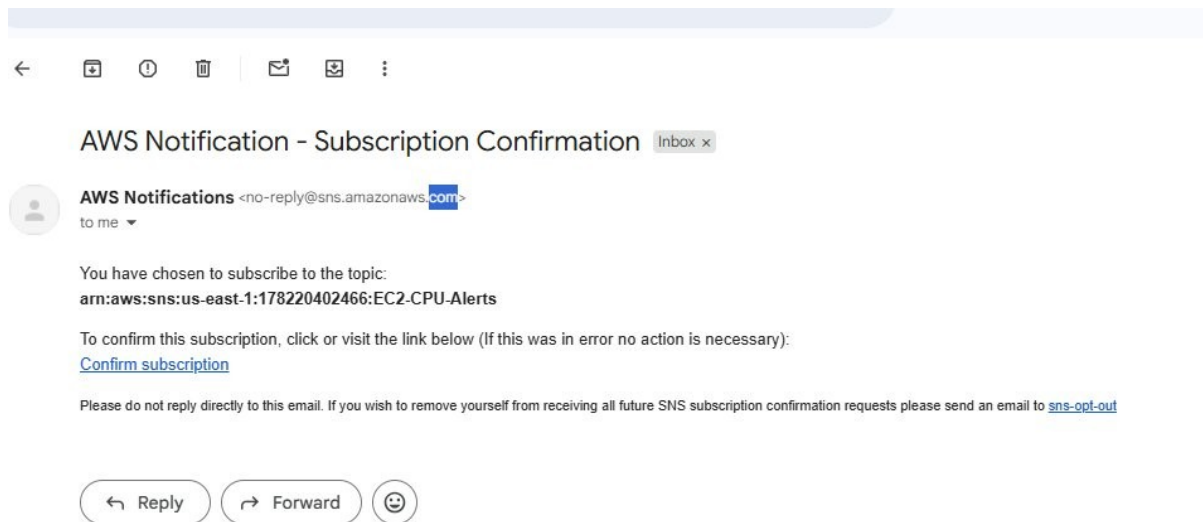


Figure 30: Subscription confirmation email from AWS Notifications

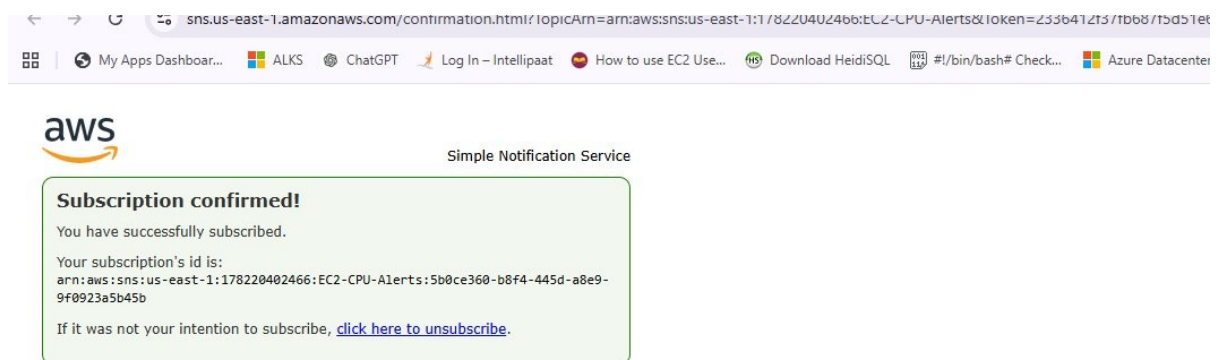


Figure 31: Subscription confirmed in the browser

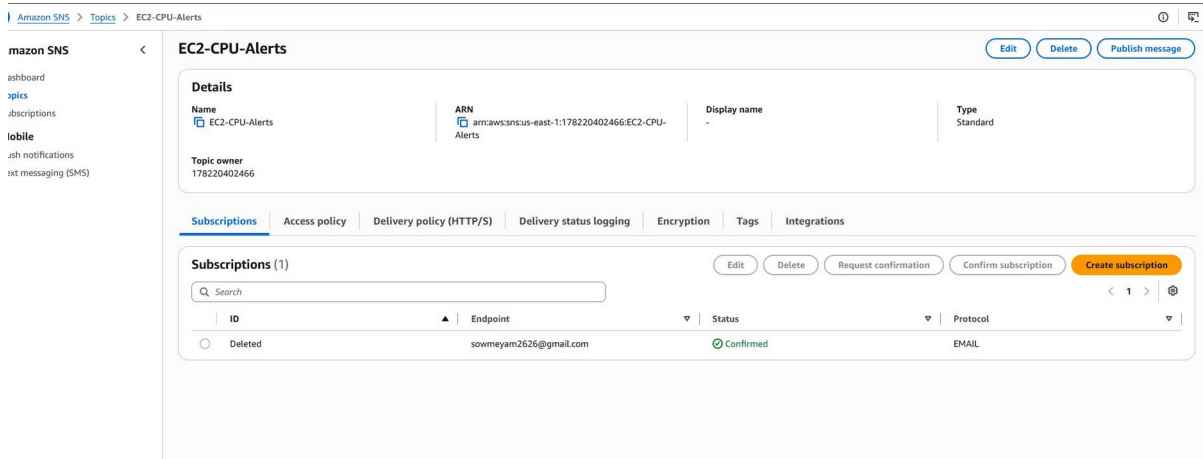


Figure 32: EC2-CPU-Alerts topic — email subscription Confirmed

Step 6: Create the CPU utilization alarm

Go to **CloudWatch** → **Alarms** → **Create alarm** → **Select metric** → **EC2** → **Per-Instance Metrics** and pick **CPUUtilization** for the instance. Configure:

- **Statistic / Period:** Average over 5 minutes
- **Threshold:** Static, Greater than 65 (%)
- **Notification:** In alarm → send to SNS topic EC2-CPU-Alerts
- **Alarm name:** EC2-CPU-Above-65

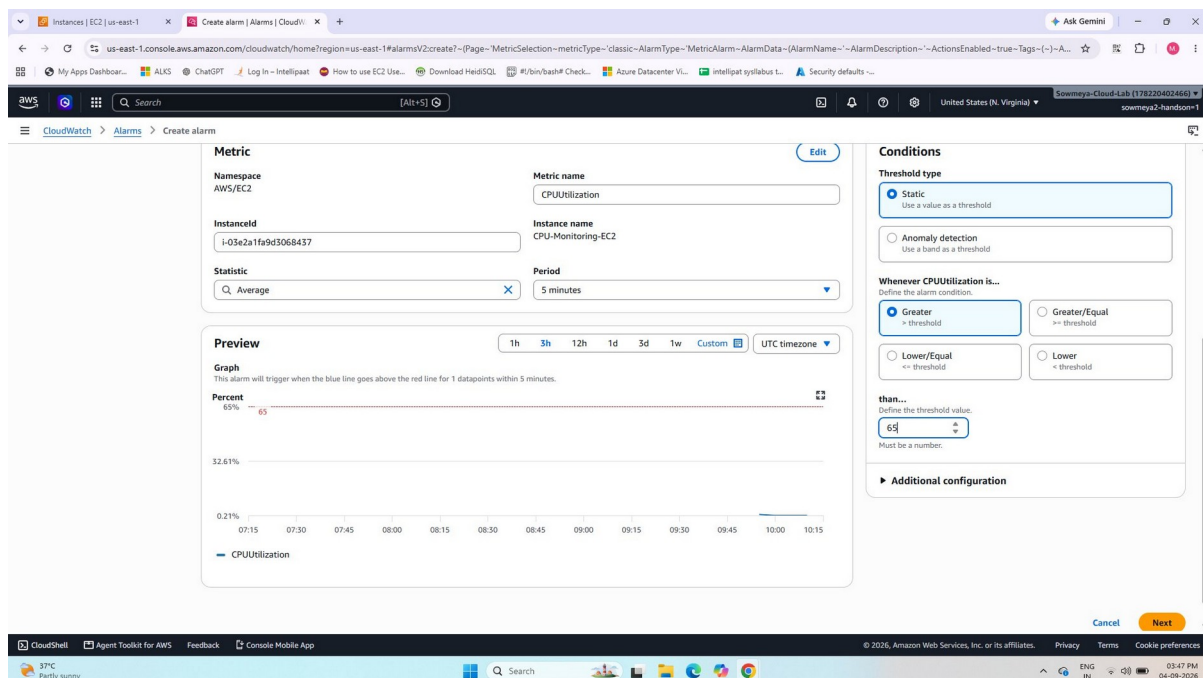


Figure 33: Alarm conditions — CPUUtilization, Average, 5 minutes, Greater than 65

Step 7: Verify the alarms and subscription

The Alarms list now shows both alarms in the OK state. EC2-CPU-Above-65 shows "Actions enabled", meaning it will notify SNS when it enters ALARM. The Subscriptions page lists the confirmed email endpoint for EC2-CPU-Alerts.

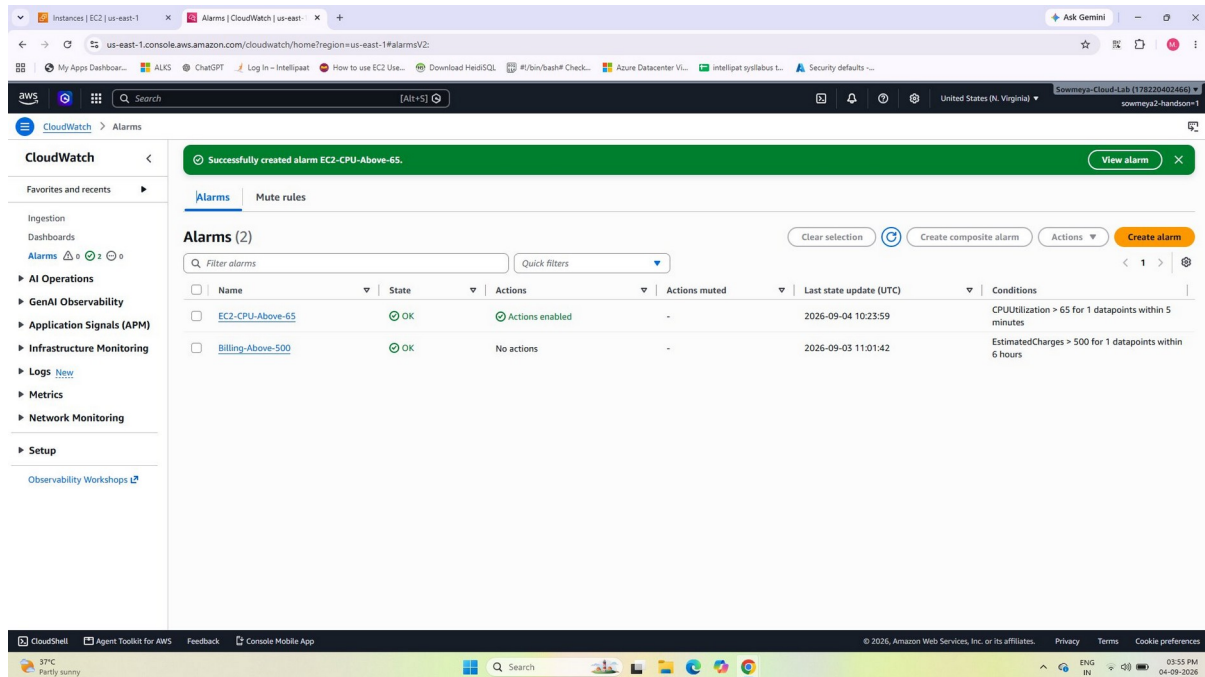


Figure 34: Alarms list — EC2-CPU-Above-65 and Billing-Above-500 both in OK state

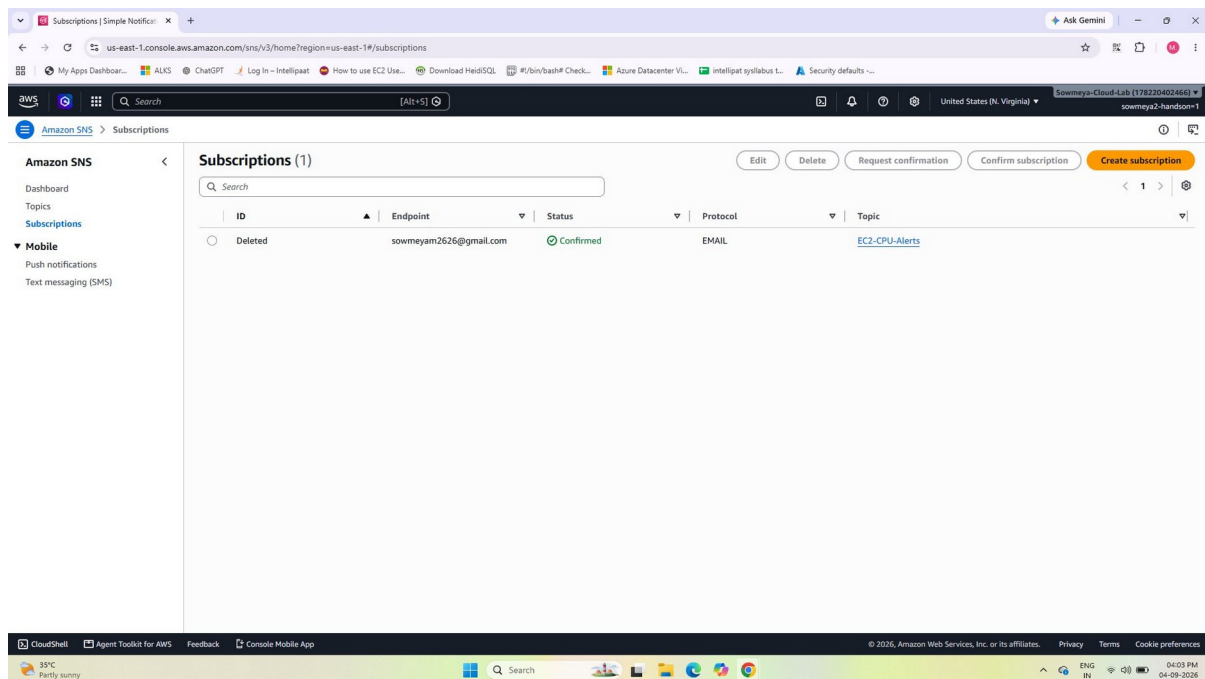


Figure 35: SNS Subscriptions — email endpoint Confirmed for EC2-CPU-Alerts

5.4 Analysis

The CPU-alarm pipeline (EC2 → CloudWatch → SNS → email) is fully configured and the subscription is confirmed. The billing alarm is also created with the correct \$500 threshold.

6. Assignment 5 — CloudWatch Dashboard

6.1 Tasks

1. Launch an EC2 instance to monitor.
2. Create a CloudWatch dashboard showing the instance's CPU utilization and network traffic.

6.2 Step-by-Step Implementation

Step 1: Launch the EC2 instance

In **Asia Pacific (Mumbai) ap-south-1**, launch a **t3.micro** Amazon Linux 2023 instance named **CW-Monitoring-EC2**. Confirm it is **Running** with 3/3 status checks passed.

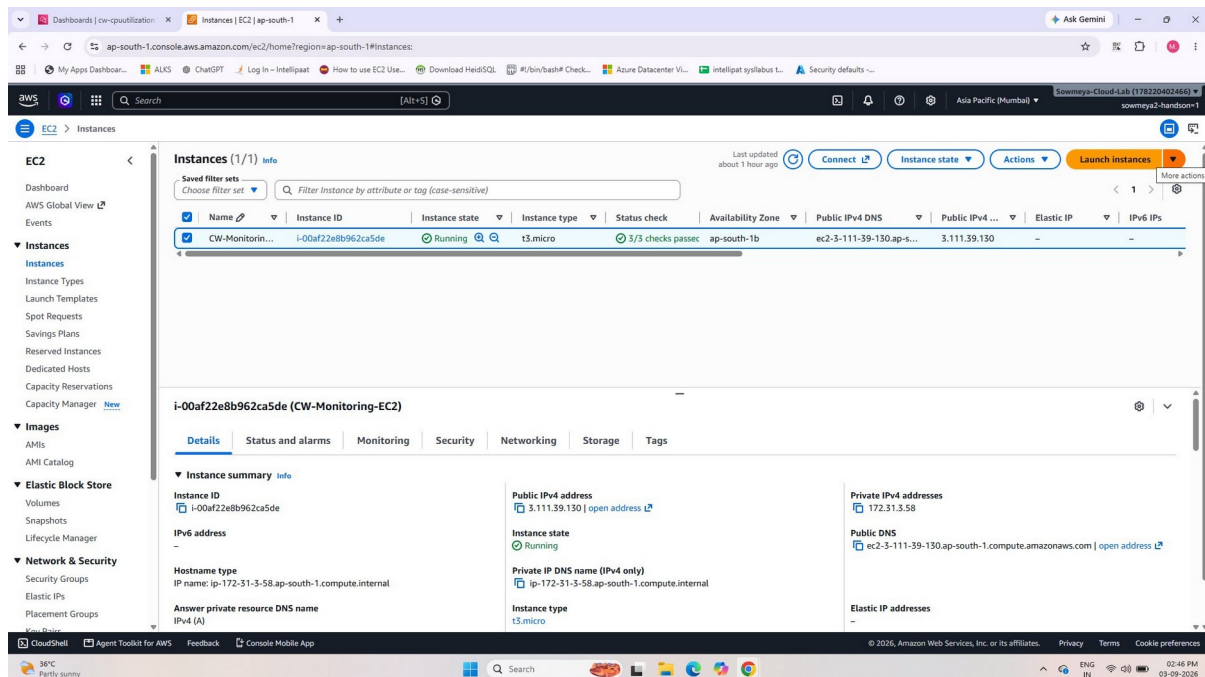


Figure 36: CW-Monitoring-EC2 running in ap-south-1b

Step 2: Review the instance details

The instance summary shows the Instance ID (i-00af22e8b962ca5de), public/private IPs, VPC and subnet, and "Monitoring: disabled" — meaning basic monitoring, where EC2 sends metrics to CloudWatch every 5 minutes at no extra cost.

Understanding: Basic monitoring (5-minute data) is free and on by default. Detailed monitoring (1-minute data) costs extra and is only needed when you must react faster.

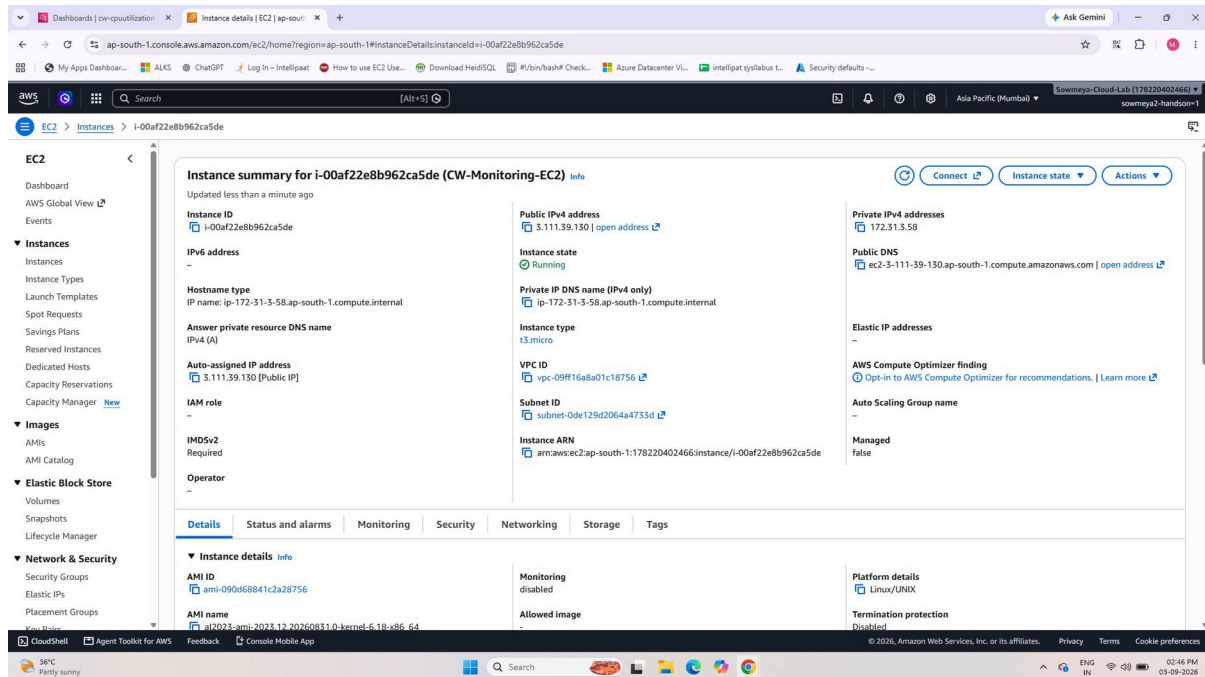


Figure 37: Instance summary — t3.micro, Amazon Linux 2023, basic monitoring

Step 3: Create the dashboard and add widgets

Go to **CloudWatch** → **Dashboards** → **Create dashboard**, name it **cw-cputilization**. Add three **Line** widgets from **EC2** → **Per-Instance Metrics** for this instance:

- **CPUUtilization** — percentage of CPU in use
- **NetworkIn** — bytes received by the instance
- **NetworkOut** — bytes sent by the instance

Click **Save** to keep the layout.

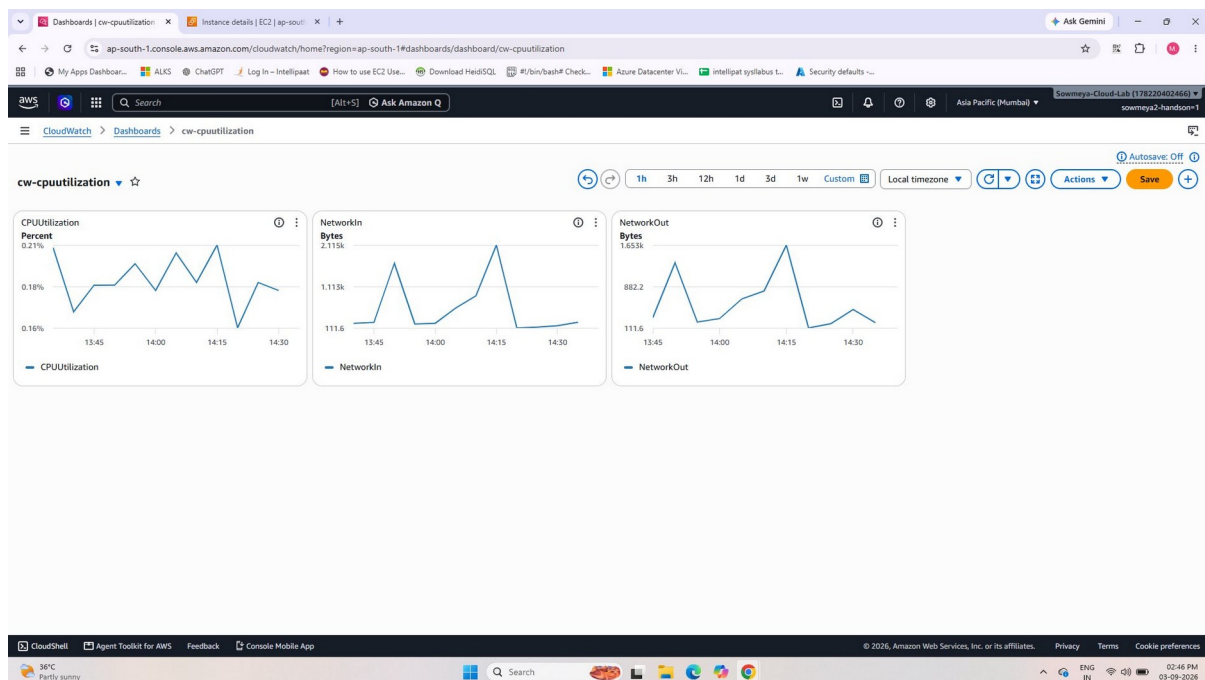


Figure 38: Dashboard cw-cputilization — CPUUtilization, NetworkIn, NetworkOut

Step 4: Refresh to confirm live data

After a few minutes, refresh the dashboard. New datapoints appear up to 14:45, showing the widgets update automatically as the instance reports metrics.

Understanding: CPU stays around 0.2% because the instance is idle. Running a load (for example the "stress" tool) would show a visible spike — useful to connect this dashboard with the alarm in Assignment 4.

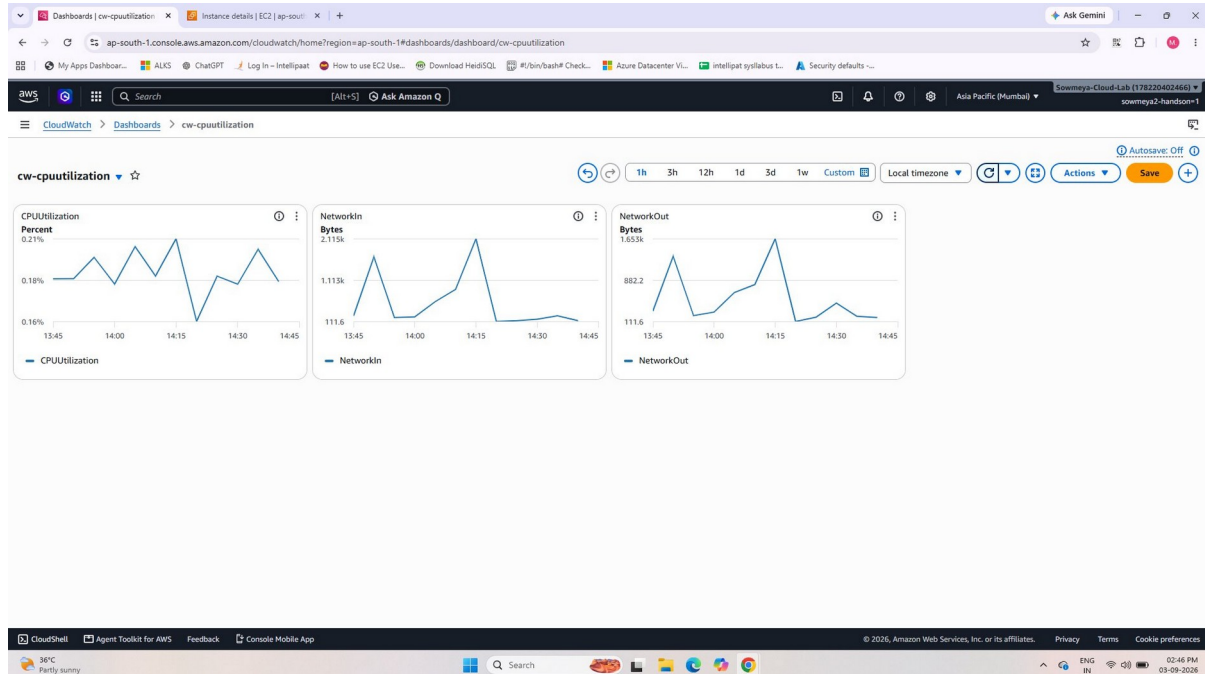


Figure 39: Refreshed dashboard — graphs extend with new datapoints

6.3 Analysis

The dashboard is created with the three key EC2 metrics and is receiving live data. Adding a widget for the EC2-CPU-Above-65 alarm, or placing both instances on one dashboard, would tie Assignments 4 and 5 together.

7. Observations & Recommendations

7.1 Good-practice notes

- **Attach the AllowAssume policy to Dev-Team, not to each user.** It was attached directly to Dev1 and Dev2. Attaching it to the Dev-Team group means new developers get it automatically. (The trust policy still has to list user ARNs, or trust the account and rely on the group policy.)
- **Enable MFA.** Dev1 and Dev2 show "Console access: Enabled without MFA". Require MFA for console users — and optionally add an "aws:MultiFactorAuthPresent" condition to the role trust policy.
- **Dev1 in both groups.** Dev1 inherits both XYZ-Policy-1 and XYZ-Policy-2. If this is intentional (a developer with ops duties), document it; otherwise remove Dev1 from Ops-Team.
- **Scope resources.** Policies use "All resources". In production, narrow them to specific buckets, instances or tables.
- **Clean up to avoid charges.** Two t3.micro instances (us-east-1 and ap-south-1) were launched for Assignments 4–5. Stop or terminate them when finished.
- **Screenshot hygiene.** The screenshots show the AWS account ID, access key prefix and a personal email address. Consider blurring these, since the repository is public on GitHub.

7.2 Conclusion

The IAM setup correctly separates developer and operations access through groups and customer-managed policies, and uses an assumable role to give developers temporary, controlled access to VPC and DynamoDB. The tests confirm both the positive path (role works) and the negative path (access denied without the role). The CloudWatch work adds visibility and alerting: a billing guardrail, a CPU alarm wired to email through SNS, and a live EC2 dashboard. All five assignments are complete.